

TFM: Impacto de la generación de embeddings en tareas con arquitectura transformer

TAREA: Clasificación multietiqueta de papers matemáticos.

Algunos modelos tardan bastante en ejecutarse. Por ello he ido guardándolos en una carpeta en Drive. Si quiere probar a ejecutar el código se recomienda antes seguir los siguientes pasos:

1. Descargarse el código y abrirlo desde Google Colab.
2. Pulsar el siguiente enlace: https://drive.google.com/drive/folders/1LIu7P33rzkW9zLDssTRN5SYB3gan27Z?usp=drive_link
3. Añadir un acceso directo de la carpeta en vuestra unidad, pulsando [Añadir a Mi unidad](#).
4. Ejecutar el código y aceptar todas las condiciones de Google (será para poder acceder a esa carpeta y utilizar los archivos).

✓ 1. LIBRERÍAS

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!pip install transformers torch
!python -m spacy download en_core_web_sm
!pip install gensim
!pip install scikit-multilearn
!pip install umap-learn
```

[Mostrar salida oculta](#)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from ast import literal_eval # Para evaluar las listas de categorias si vienen como string
import json
import os
import re
import joblib
from time import time
from tqdm import tqdm # para barra de progreso
import warnings
warnings.filterwarnings('ignore') # Ocultar warnings de convergencia
import gc
import spacy
from collections import Counter
from datasets import Dataset
import random
import psutil

from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
from skmultilearn.model_selection import iterative_train_test_split
from sklearn.preprocessing import Normalizer

# Para la extracción de características (Embeddings)
from sklearn.feature_extraction.text import TfidfVectorizer
from gensim.models import Word2Vec
from sklearn.decomposition import TruncatedSVD

# Para los modelos de clasificación
from sklearn.dummy import DummyClassifier
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
from sklearn.kernel_approximation import Nystroem
from sklearn.pipeline import make_pipeline

# Para la evaluación
from sklearn.metrics import (
```

```

        accuracy_score,
        classification_report,
        f1_score,
        roc_auc_score,
        average_precision_score
    )

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from sklearn.metrics import classification_report, f1_score
from tensorflow.keras.models import load_model
from gensim.models import Word2Vec
import scipy.special
import torch
from transformers import AutoTokenizer, AutoModel, AutoModelForSequenceClassification, TrainingArguments, Trainer, EvalPrece

import umap
from sklearn.manifold import TSNE

# Rutas de la base de datos de DRIVE:
json_file_path = '/content/drive/MyDrive/TFM_Carolina_Herrera/arxiv-metadata-oai-snapshot.json'
# Ruta de la base de datos preprocesada:
final_drive_path = '/content/drive/MyDrive/TFM_Carolina_Herrera/math_papers_pure_cleaned.csv'
# Cargar diccionario GloVe
EMBEDDING_FILE_GLOVE = '/content/drive/MyDrive/TFM_Carolina_Herrera/glove.6B.100d.txt'
# Guardar los modelos:
save_dir_base = '/content/drive/MyDrive/TFM_Carolina_Herrera/modelos_matematicos/'
# Guardar los embeddings de test y train:
drive_dir = '/content/drive/MyDrive/TFM_Carolina_Herrera/embeddings_guardados/'

# Lista para acumular todos los resultados y compararlos al final
resultados_globales = []

```

```

os.makedirs(drive_dir, exist_ok=True)
os.makedirs(save_dir_base, exist_ok=True)

```

```

def visualizar_espacio_latente(X, Y, clases, titulo, n_samples=500, metodo='umap'):
    """
    X: Matriz de embeddings (ej. X_test_w2v)
    Y: Matriz binaria de etiquetas (y_test)
    clases: Lista con los nombres de las categorías
    """
    # Papers con solo 1 categoría
    indices_una_etiqueta = np.where(Y.sum(axis=1) == 1)[0]
    n_tomar = min(n_samples, len(indices_una_etiqueta))

    # Seleccionar aleatoriamente
    np.random.seed(42)
    indices_muestra = np.random.choice(indices_una_etiqueta, size=n_tomar, replace=False)

    X_muestra = X[indices_muestra]
    Y_muestra = Y[indices_muestra]

    # Traducir el vector binario [0, 1, 0...] al nombre de la clase
    etiquetas_texto = [clases[np.argmax(fila)] for fila in Y_muestra]

    # Reducción de dimensionalidad a 2D con UMAP y T-SNE
    if metodo == 'tsne':
        # PCA previo ayuda a t-SNE
        reductor = TSNE(n_components=2, perplexity=30, random_state=42, init='pca', learning_rate='auto')
    elif metodo == 'umap':
        reductor = umap.UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42)
    else:
        raise ValueError("El método debe ser 'tsne' o 'umap'")

    X_2d = reductor.fit_transform(X_muestra)

    # Graficar
    plt.figure(figsize=(12, 8))
    sns.scatterplot(
        x=X_2d[:, 0],
        y=X_2d[:, 1],
        hue=etiquetas_texto,
        palette="tab20",
        s=80,
        alpha=0.8,
        edgecolor='k'
    )

```

```
#plt.title(f"{{titulo}} ({{metodo.upper()}})", fontsize=14)
#plt.xlabel("Dimensión 1")
#plt.ylabel("Dimensión 2")
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', title="Categorías", fontsize=9)
plt.tight_layout()
plt.show()
```

```
def extract_finetuned_embeddings(text_series, model, tokenizer, batch_size=32, max_len=256):
    embeddings = []
    text_list = text_series.tolist()

    for i in tqdm(range(0, len(text_list), batch_size), desc="Extrayendo Embeddings"):
        batch_texts = text_list[i:i + batch_size]

        # Tokenización
        inputs = tokenizer(
            batch_texts,
            padding="max_length",
            truncation=True,
            max_length=max_len,
            return_tensors="pt"
        ).to(device)

        with torch.no_grad():
            outputs = model(**inputs)

            # outputs.hidden_states es una tupla con las salidas de todas las capas.
            # Tomamos la última capa [-1]
            last_hidden_state = outputs.hidden_states[-1]

            # De toda la frase, nos quedamos solo con el token [CLS] (posición 0)
            cls_embeddings = last_hidden_state[:, 0, :].cpu().numpy().astype(np.float32)
            embeddings.append(cls_embeddings)

        # Limpieza de memoria GPU (muy importante para evitar OOM)
        del inputs, outputs, last_hidden_state, cls_embeddings
        torch.cuda.empty_cache()

    return np.vstack(embeddings)
```

```
def extract_t5_embeddings(text_series, model, tokenizer, batch_size=32, max_len=256):
    embeddings = []
    text_list = text_series.tolist()

    for i in tqdm(range(0, len(text_list), batch_size), desc="Extrayendo T5"):
        batch_texts = text_list[i:i + batch_size]

        inputs = tokenizer(
            batch_texts,
            padding="max_length",
            truncation=True,
            max_length=max_len,
            return_tensors="pt"
        ).to(device)

        with torch.no_grad():
            # Pasamos los textos solo por el Encoder de T5
            encoder_outputs = model.get_encoder()(**inputs)
            last_hidden_state = encoder_outputs.last_hidden_state

            # Hacemos Mean Pooling (promedio) ignorando los tokens de padding (ceros)
            attention_mask = inputs['attention_mask'].unsqueeze(-1).expand(last_hidden_state.size()).float()
            sum_embeddings = torch.sum(last_hidden_state * attention_mask, 1)
            sum_mask = attention_mask.sum(1)
            mean_pooled = sum_embeddings / torch.clamp(sum_mask, min=1e-9)

            embeddings.append(mean_pooled.cpu().numpy().astype(np.float32))

        del inputs, encoder_outputs, last_hidden_state, mean_pooled, attention_mask
        torch.cuda.empty_cache()

    return np.vstack(embeddings)
```

```
# Función auxiliar para obtener la memoria RAM actual en MB
def get_memory():
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / (1024 ** 2)
```

2. BASE DE DATOS (importar y preprocesado)

> NOTAS

Procesado de datos para modelos clasicos

- Elimino todas las ecuaciones.
- Elimino todos los símbolos que no sean letras
- Todo en minúscula
- Lemmatizo
- Elimino StopWords

Procesado para modelos tranformer

- Sustituyo las ecuaciones por `math_token`.
- Lemmatizo
- Elimino StopWords

Procesado general

- Elimino NA
- Elimino duplicados

↳ 1 celda oculta

✓ EDA

```
print(f"Dimensiones del dataset: {df_math.shape}")
df_math.info()
```

```
Dimensiones del dataset: (468883, 6)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 468883 entries, 0 to 468882
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    468883 non-null object
1   categories            468883 non-null object
2   categories_list       468883 non-null object
3   abstract              468883 non-null object
4   clean                 468883 non-null object
5   abstract_transformer  468883 non-null object
dtypes: object(6)
memory usage: 21.5+ MB
```

```
# Calculamos cuántas etiquetas tiene cada paper
df_math['num_labels'] = df_math['categories_list'].apply(len)
print(df_math['num_labels'].describe())
```

```
count    468883.000000
mean         1.378905
std         0.625044
min         1.000000
25%         1.000000
50%         1.000000
75%         2.000000
max         7.000000
Name: num_labels, dtype: float64
```

```
# Aplanamos la lista de listas en una sola gran lista
all_labels = [label for sublist in df_math['categories_list'] for label in sublist]

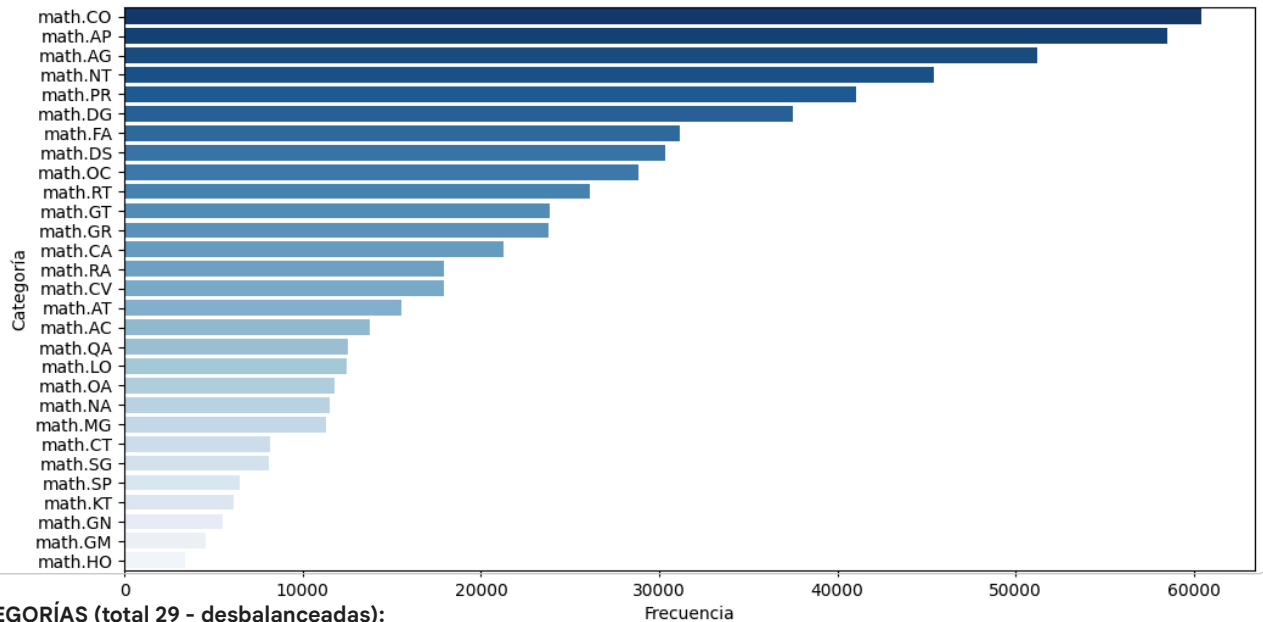
# Contamos la frecuencia de cada etiqueta
label_counts = Counter(all_labels)

# Convertimos a DataFrame
df_label_counts = pd.DataFrame(label_counts.items(), columns=['Categoria', 'Frecuencia']).sort_values(by='Frecuencia', ascending=False)

print(f"Total de categorías únicas: {len(df_label_counts)}")

# Graficamos
plt.figure(figsize=(12, 6))
sns.barplot(data=df_label_counts, y='Categoria', x='Frecuencia', palette='Blues_r')
plt.xlabel('Frecuencia')
plt.ylabel('Categoría')
plt.show()
```

Total de categorías únicas: 29



CATEGORÍAS (total 29 - desbalanceadas):

- math.CO (Combinatorics): Combinatoria. Estudio de estructuras discretas, teoría de grafos, conteo y optimización discreta.
- math.AP (Analysis of PDEs): Análisis de Ecuaciones en Derivadas Parciales.
- math.AG (Algebraic Geometry): Geometría Algebraica.
- math.NT (Number Theory): Teoría de Números.
- math.PR (Probability): Teoría de la probabilidad, procesos estocásticos y sus aplicaciones.
- math.DG (Differential Geometry): Geometría Diferencial.
- math.FA (Functional Analysis): Análisis funcional.
- math.DS (Dynamical Systems): Sistemas dinámicos, sistemas geométricos, ecuaciones diferenciales.
- math.OC (Optimization and Control): Investigación operativa, control óptimo y minimización/maximización de funciones.
- math.RT (Representation Theory): Estructuras algebraicas a través de representaciones lineales.
- math.GT (Geometry Topology): Geometría topológica.
- math.GR (Group Theory): Teoría de grupos.
- math.CA (Classical Analysis): Análisis matemático.
- math.RA (Rings and Algebras): Álgebra.
- math.CV (Complex Variables): Análisis complejo.
- math.AT (Algebraic Topology): Topología algebraica (homología, homotopía...)
- math.AC (Commutative Algebra): Álgebra conmutativa.
- math.QA (Quantum Algebra): Álgebra cuántica (grupos cuánticos, invariantes inspiradas en física cuántica...)
- math.LO (Logic): Lógica.
- math.OA (Operator Algebras): Álgebras de operadores (operadores de Von Neumann...)
- math.NA (Numerical Analysis): Análisis Numérico.

- math.MG (Metric Geometry): espacios métricos, geometría de distancias...
- math.CT (Category Theory): Teoría de categorías.
- math.SG (Symplectic Geometry): Geometría detrás de la mecánica clásica y sistemas hamiltonianos.
- math.SP (Spectral Theory): autovalores y espectros de operadores.
- math.KT (K-Theory): Herramientas algebraicas para estudiar fibrados vectoriales y topología.
- math.GN (General Topology): Topología.
- math.GM (General Mathematics): temas que no encajan en una categoría específica.
- math.HO (History and Overview): Historia de las matemáticas, biografías, textos educativos...

(Esta información se encuentra en: https://arxiv.org/category_taxonomy)

3. PROCESADO DE DATOS

BINARIZADO

```
# Creamos una matriz Y (variable objetivo) donde cada fila es un paper
# y cada columna una categoría. Es una matriz formada por 0 o 1.

mlb = MultiLabelBinarizer()
Y = mlb.fit_transform(df_math['categories_list'])

# Extraemos las clases que el modelo ha detectado (el vocabulario de etiquetas)
clases_matematicas = mlb.classes_

print(f"Dimensión de Y: {Y.shape}")
print(f"Total de categorías detectadas: {len(clases_matematicas)}\n")
```

```
Dimensión de Y: (468883, 29)
Total de categorías detectadas: 29
```

TRAIN Y TEST

- Separamos entrenamiento y test de forma estratificada.
- Al ser un problema multietiqueta no podemos hacerlo con: `StratifiedKFold` o `stratify=y` en la función `train_test_split` de scikit-learn, ya que no es posible contar todas las etiquetas y sacar 80% y 20% de cada una.
- Usamos `iterative_train_test_split` de `scikit-multilearn`:
 1. El algoritmo busca la categoría menos frecuente.
 2. Divide los papers de esa etiqueta en 80/20.
 3. Revisa qué otras etiquetas acompañaban a esos papers y actualiza la cuenta de lo que falta por repartir.
 4. Pasa a la segunda etiqueta más rara y repite. Así con todas.

```
# CON TODA LA BASE DE DATOS ENTERA:
df_50k = df_math.copy()
Y_50k = Y.copy()
```

```
print("--- DIVISIÓN TRAIN / TEST ---")

ruta_y_train = os.path.join(drive_dir, 'y_train.npy')
ruta_y_test = os.path.join(drive_dir, 'y_test.npy')
ruta_indices_train = os.path.join(drive_dir, 'indices_train.npy')
ruta_indices_test = os.path.join(drive_dir, 'indices_test.npy')

# Comprobamos si el split ya está calculado y guardado
if (os.path.exists(ruta_y_train) and os.path.exists(ruta_y_test) and
    os.path.exists(ruta_indices_train) and os.path.exists(ruta_indices_test)):

    print("Cargando división de Train y Test desde Drive.")
    y_train = np.load(ruta_y_train)
    y_test = np.load(ruta_y_test)
    indices_train = np.load(ruta_indices_train)
    indices_test = np.load(ruta_indices_test)

else:
    print("Calculando estratificación iterativa...")
    np.random.seed(42)
```

```
# Creamos una matriz de índices 2D para que la librería la acepte
indices = np.arange(df_50k.shape[0]).reshape(-1, 1)

# Aplicamos la estratificación iterativa
indices_train, y_train, indices_test, y_test = iterative_train_test_split(indices, Y_50k, test_size=0.2)

# Aplanamos los índices para poder usarlos en Pandas
indices_train = indices_train.flatten()
indices_test = indices_test.flatten()

# Guardamos los índices y las etiquetas en Drive para futuras ejecuciones
np.save(ruta_y_train, y_train)
np.save(ruta_y_test, y_test)
np.save(ruta_indices_train, indices_train)
np.save(ruta_indices_test, indices_test)
print("Índices y etiquetas guardadas en Drive exitosamente.")

# Separamos los papers (esto es súper rápido usando los índices ya cargados)
X_train_clean = df_50k['clean'].iloc[indices_train].astype(str)
X_test_clean = df_50k['clean'].iloc[indices_test].astype(str)

X_train_trans = df_50k['abstract_transformer'].iloc[indices_train].astype(str)
X_test_trans = df_50k['abstract_transformer'].iloc[indices_test].astype(str)

# X_train_original = df_50k['abstract'].iloc[indices_train].astype(str)
# X_test_original = df_50k['abstract'].iloc[indices_test].astype(str)

print(f"\nTamaño de Entrenamiento: {X_train_clean.shape[0]} ({round(X_train_clean.shape[0]/468883*100)}%)")
print(f"Tamaño de Test: {X_test_clean.shape[0]} ({round(X_test_clean.shape[0]/468883*100)}%)")
```

```
--- DIVISIÓN TRAIN / TEST ---
Cargando división de Train y Test desde Drive.
```

```
Tamaño de Entrenamiento: 375713 (80%)
Tamaño de Test: 93170 (20%)
```

4. EMBEDDINGS

CARGAR EMBEDDINGS

TF-IDF

$$\text{tf-idf}(t, d) = \text{tf}_{t,d} \cdot \text{idf}_t$$

- tf → la frecuencia del término:

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d) & \text{si } \text{count}(t, d) > 0 \\ 0 & \text{en otro caso} \end{cases}$$

- idf → la frecuencia inversa del documento:

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

donde N es el número total de documentos en la colección, y df_t es el número de documentos en los que aparece el término t . Cuantos menos documentos aparezcan, mayor será este peso. Los términos que ocurren en solo unos pocos documentos son útiles para discriminar

- Ya está normalizado

```
import scipy.sparse

# Cambiamos la extensión a .npz (el formato para matrices dispersas)
ruta_train_tfidf = os.path.join(drive_dir, 'X_train_tfidf.npz')
ruta_test_tfidf = os.path.join(drive_dir, 'X_test_tfidf.npz')

# Comprobamos si los archivos ya existen
if (os.path.exists(ruta_train_tfidf) and os.path.exists(ruta_test_tfidf)):

    print("--- CARGANDO MATRICES TF-IDF DESDE DRIVE ---")
    # Usamos load_npz en lugar de np.load
    X_train_tfidf = scipy.sparse.load_npz(ruta_train_tfidf)
    X_test_tfidf = scipy.sparse.load_npz(ruta_test_tfidf)

    print(f"Dimensiones X_train_tfidf: {X_train_tfidf.shape}")

    t_emb_tfidf = 1.35 #EDITADAS POR MÍ
    mem_emb_tfidf = 28119.14
```

```

else:
    print("--- ENTRENANDO TF-IDF ---")
    t0_emb = time()
    mem0_emb = get_memory()

    # Inicializamos y entrenamos el vectorizador
    tfidf_vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))

    # Quitamos el .toarray() para mantener la matriz comprimida
    X_train_tfidf = tfidf_vectorizer.fit_transform(X_train_clean)
    X_test_tfidf = tfidf_vectorizer.transform(X_test_clean)

    t_emb_tfidf = round((time() - t0_emb)/60, 2)
    mem_emb_tfidf = round(get_memory() - mem0_emb, 2)
    print(f"TF-IDF listo en {t_emb_tfidf} minutos.")
    print(f"Memoria: {mem_emb_tfidf}")
    print(f"Dimensiones X_train_tfidf: {X_train_tfidf.shape}")

    # Usamos save_npz en lugar de np.save
    scipy.sparse.save_npz(ruta_train_tfidf, X_train_tfidf)
    scipy.sparse.save_npz(ruta_test_tfidf, X_test_tfidf)

--- CARGANDO MATRICES TF-IDF DESDE DRIVE ---
Dimensiones X_train_tfidf: (375713, 10000)

```

Word2Vec

- Elegimos la técnica de promedio para representar el paper en un solo vector.

```

#ruta_w2v_model = os.path.join(save_dir_base, 'w2v_custom.model')
ruta_train_w2v = os.path.join(drive_dir, 'X_train_w2v.npy')
ruta_test_w2v = os.path.join(drive_dir, 'X_test_w2v.npy')
ruta_train_w2v_scaled = os.path.join(drive_dir, 'X_train_w2v_scaled.npy')
ruta_test_w2v_scaled = os.path.join(drive_dir, 'X_test_w2v_scaled.npy')

# Comprobamos si los archivos ya existen
if (os.path.exists(ruta_train_w2v) and
    os.path.exists(ruta_test_w2v) and
    os.path.exists(ruta_train_w2v_scaled) and
    os.path.exists(ruta_test_w2v_scaled)):

    print("--- CARGANDO MATRICES Word2Vec DESDE DRIVE ---")

    # Cargamos el modelo
    #w2v_model = Word2Vec.load(ruta_w2v_model)

    # Cargamos las matrices
    X_train_w2v = np.load(ruta_train_w2v)
    X_test_w2v = np.load(ruta_test_w2v)
    X_train_w2v_scaled = np.load(ruta_train_w2v_scaled)
    X_test_w2v_scaled = np.load(ruta_test_w2v_scaled)

    t_emb_w2v = 3.78 # EDITADO POR MÍ
    mem_emb_w2v = 3819.23

    print(f"Dimensiones X_train_w2v: {X_train_w2v.shape}")

else:
    print("--- ENTRENANDO WORD2VEC ---")
    t0_emb = time()
    mem0_emb = get_memory()

    # Tokenización
    X_train_tokens = [text.split() for text in X_train_clean]
    X_test_tokens = [text.split() for text in X_test_clean]

    # Entrenar modelo Word2Vec: vectores de dimensión 300. Ventana de 5 palabras. Palabras que aparecen >5 veces.
    w2v_model = Word2Vec(sentences=X_train_tokens,
                        vector_size=300,
                        window=5,
                        min_count=5,
                        workers=4)

    # Entrenamos un TF-IDF solo para extraer la métrica IDF de cada palabra
    tfidf_w2v = TfidfVectorizer(
        tokenizer=lambda x: x.split(), # Usamos el mismo split() que en Word2Vec
        token_pattern=None,           # Desactivamos el patrón por defecto para que no interfiera
        lowercase=False
    )

```



```

# Entrenamos
tfidf_w2v.fit(X_train_clean)

# Creamos el diccionario de pesos
word2weight = {word: tfidf_w2v.idf_[i] for word, i in tfidf_w2v.vocabulary_.items()}

# Verifica que los tokens están en el diccionario y promedia los vectores para representar el paper
def document_vector_weighted(word2vec_model, doc_tokens, weight_dict):
    # Filtramos palabras que estén tanto en Word2Vec como en nuestro diccionario de pesos
    valid_words = [word for word in doc_tokens if word in word2vec_model.wv.key_to_index and word in weight_dict]
    if not valid_words:
        return np.zeros(word2vec_model.vector_size)

    # Extraemos los vectores de las palabras válidas
    vectors = word2vec_model.wv[valid_words]
    # Extraemos los pesos correspondientes a esas palabras
    weights = np.array([weight_dict[word] for word in valid_words])
    # Calculamos el promedio ponderado (suma(vector * peso) / suma(pesos))
    weighted_avg = np.average(vectors, axis=0, weights=weights)

    return weighted_avg

# Generar los embeddings ponderados
X_train_w2v = np.array([document_vector_weighted(w2v_model, doc, word2weight) for doc in X_train_tokens])
X_test_w2v = np.array([document_vector_weighted(w2v_model, doc, word2weight) for doc in X_test_tokens])

# Normalizamos
scaler = Normalizer(norm='l2')
X_train_w2v_scaled = scaler.fit_transform(X_train_w2v)
X_test_w2v_scaled = scaler.transform(X_test_w2v)

t_emb_w2v = round((time() - t0_emb)/60, 2)
mem_emb_w2v = round(get_memory() - mem0_emb, 2)

print(f"Word2Vec listo en {t_emb_w2v} minutos.")
print(f"Memoria: {mem_emb_w2v}")
print(f"Dimensiones X_train_w2v: {X_train_w2v.shape}")

# Guardar matrices de Word2Vec
np.save(ruta_train_w2v, X_train_w2v)
np.save(ruta_test_w2v, X_test_w2v)
np.save(ruta_train_w2v_scaled, X_train_w2v_scaled)
np.save(ruta_test_w2v_scaled, X_test_w2v_scaled)

--- CARGANDO MATRICES Word2Vec DESDE DRIVE ---
Dimensiones X_train_w2v: (375713, 300)

```

▼ GloVe

```

ruta_train_glove = os.path.join(drive_dir, 'X_train_glove.npy')
ruta_test_glove = os.path.join(drive_dir, 'X_test_glove.npy')
ruta_train_glove_scaled = os.path.join(drive_dir, 'X_train_glove_scaled.npy')
ruta_test_glove_scaled = os.path.join(drive_dir, 'X_test_glove_scaled.npy')

# Comprobamos si las matrices ya existen
if (os.path.exists(ruta_train_glove) and
    os.path.exists(ruta_test_glove) and
    os.path.exists(ruta_train_glove_scaled) and
    os.path.exists(ruta_test_glove_scaled)):

    print("--- CARGANDO MATRICES GLOVE DESDE DRIVE ---")

    # Cargamos las matrices
    X_train_glove = np.load(ruta_train_glove)
    X_test_glove = np.load(ruta_test_glove)
    X_train_glove_scaled = np.load(ruta_train_glove_scaled)
    X_test_glove_scaled = np.load(ruta_test_glove_scaled)
    print(f"Dimensiones X_train_glove: {X_train_glove.shape}")

    t_emb_glove = 0.78
    mem_emb_glove = 1840.8

else:
    print("--- ENTRENANDO GLOVE ---")

    t0_emb = time()
    mem0_emb = get_memory()

    # Cargar el diccionario de GloVe
    embed_size = 100 # dimension del vector, es el modelo que he elegido
    def get_coefs(word, *arr):

```

```

        return word, np.asarray(arr, dtype='float32')
embeddings_index = dict(get_coefs(*o.rstrip().rsplit(' ')) for o in open(EMBEDDING_FILE_GLOVE, encoding='utf-8'))

# Separamos las palabras (tokens)
X_train_tokens_glove = [text.split() for text in X_train_clean]
X_test_tokens_glove = [text.split() for text in X_test_clean]

# Promediar
def document_vector_glove(doc_tokens):
    valid_words = [word for word in doc_tokens if word in embeddings_index]
    if not valid_words:
        return np.zeros(embed_size)
    vectors = [embeddings_index[word] for word in valid_words]
    return np.mean(vectors, axis=0)

# Generar matrices
X_train_glove = np.array([document_vector_glove(doc) for doc in X_train_tokens_glove])
X_test_glove = np.array([document_vector_glove(doc) for doc in X_test_tokens_glove])

# Escalamos
scaler = Normalizer(norm='l2')
X_train_glove_scaled = scaler.fit_transform(X_train_glove)
X_test_glove_scaled = scaler.transform(X_test_glove)

t_emb_glove = round((time() - t0_emb)/60, 2)
mem_emb_glove = round(get_memory() - mem0_emb, 2)

print(f"GloVe listo en {t_emb_glove} minutos.")
print(f"Memoria: {mem_emb_glove}.")
print(f"Dimensiones X_train_glove: {X_train_glove.shape}")

# Guardar matrices de GloVe
np.save(ruta_train_glove, X_train_glove)
np.save(ruta_test_glove, X_test_glove)
np.save(ruta_train_glove_scaled, X_train_glove_scaled)
np.save(ruta_test_glove_scaled, X_test_glove_scaled)

--- CARGANDO MATRICES GLOVE DESDE DRIVE ---
Dimensiones X_train_glove: (375713, 100)

```

✓ SciBERT

- Elegí SciBERT por haber sido entrenado con papers. Pero sin hacer fine-tuning los resultados no eran muy buenos.

```

ruta_train_bert = os.path.join(drive_dir, 'X_train_bert.npy')
ruta_test_bert = os.path.join(drive_dir, 'X_test_bert.npy')
ruta_train_bert_scaled = os.path.join(drive_dir, 'X_train_bert_scaled.npy')
ruta_test_bert_scaled = os.path.join(drive_dir, 'X_test_bert_scaled.npy')

# Comprobamos si las matrices ya existen en drive
if (os.path.exists(ruta_train_bert) and
    os.path.exists(ruta_test_bert) and
    os.path.exists(ruta_train_bert_scaled) and
    os.path.exists(ruta_test_bert_scaled)):

    print("--- CARGANDO MATRICES SCIBERT DESDE DRIVE ---")

    # Cargamos las matrices directamente
    X_train_bert = np.load(ruta_train_bert)
    X_test_bert = np.load(ruta_test_bert)
    X_train_bert_scaled = np.load(ruta_train_bert_scaled)
    X_test_bert_scaled = np.load(ruta_test_bert_scaled)

    t_emb_bert = 22.37
    mem_emb_bert = 1739.41

    print(f"Dimensiones X_train_bert_scaled: {X_train_bert_scaled.shape}")

else:
    print("--- ENTRENANDO SCIBERT ---")

    t0_emb = time()
    mem0_emb = get_memory()

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # Usar GPU si la hay
    print(f"Dispositivo en uso: {device}")

    # Descarga del modelo
    model_name = "allenai/scibert_scivocab_uncased"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model_bert = AutoModel.from_pretrained(model_name).to(device)

```

```

# Ponemos 256 tokens para no saturar la RAM
def get_bert_embeddings(text_series, batch_size=32, max_len=256):
    model_bert.eval()
    embeddings = []
    text_list = text_series.tolist()

    for i in tqdm(range(0, len(text_list), batch_size), desc="Extrayendo..."):
        batch_texts = text_list[i : i + batch_size]

        inputs = tokenizer(batch_texts, padding=True, truncation=True,
                           max_length=max_len, return_tensors="pt").to(device)

        with torch.no_grad(): # apagar el motor del cálculo del gradiente
            outputs = model_bert(**inputs)

        # token [CLS]
        cls_embeddings = outputs.last_hidden_state[:, 0, :].cpu().numpy().astype(np.float32) #Convertir directamente a
        embeddings.append(cls_embeddings)

        # Limpiar variables temporales de la GPU en cada iteración
        del inputs, outputs, cls_embeddings
        torch.cuda.empty_cache()

    return np.vstack(embeddings).astype(np.float32)

print("\nExtrayendo SciBERT para Entrenamiento.")
X_train_bert = get_bert_embeddings(X_train_trans, batch_size=32)

print("\nExtrayendo SciBERT para Test.")
X_test_bert = get_bert_embeddings(X_test_trans, batch_size=32)

# Borrar el modelo
del model_bert
torch.cuda.empty_cache()
gc.collect() # Forzamos a Python a limpiar la RAM general

scaler = Normalizer(norm='l2')

# Forzamos a float32
X_train_bert_scaled = scaler.fit_transform(X_train_bert).astype(np.float32)
X_test_bert_scaled = scaler.transform(X_test_bert).astype(np.float32)

t_emb_bert = round((time() - t0_emb)/60, 2)
mem_emb_bert = round(get_memory() - mem0_emb, 2)
print(f"SciBERT listo en {t_emb_bert} minutos.")
print(f"Memoria: {mem_emb_bert}")
print(f"Dimensiones X_train_bert_scaled: {X_train_bert_scaled.shape}")

# Guardar matrices de SciBERT
np.save(ruta_train_bert, X_train_bert)
np.save(ruta_test_bert, X_test_bert)
np.save(ruta_train_bert_scaled, X_train_bert_scaled)
np.save(ruta_test_bert_scaled, X_test_bert_scaled)

```

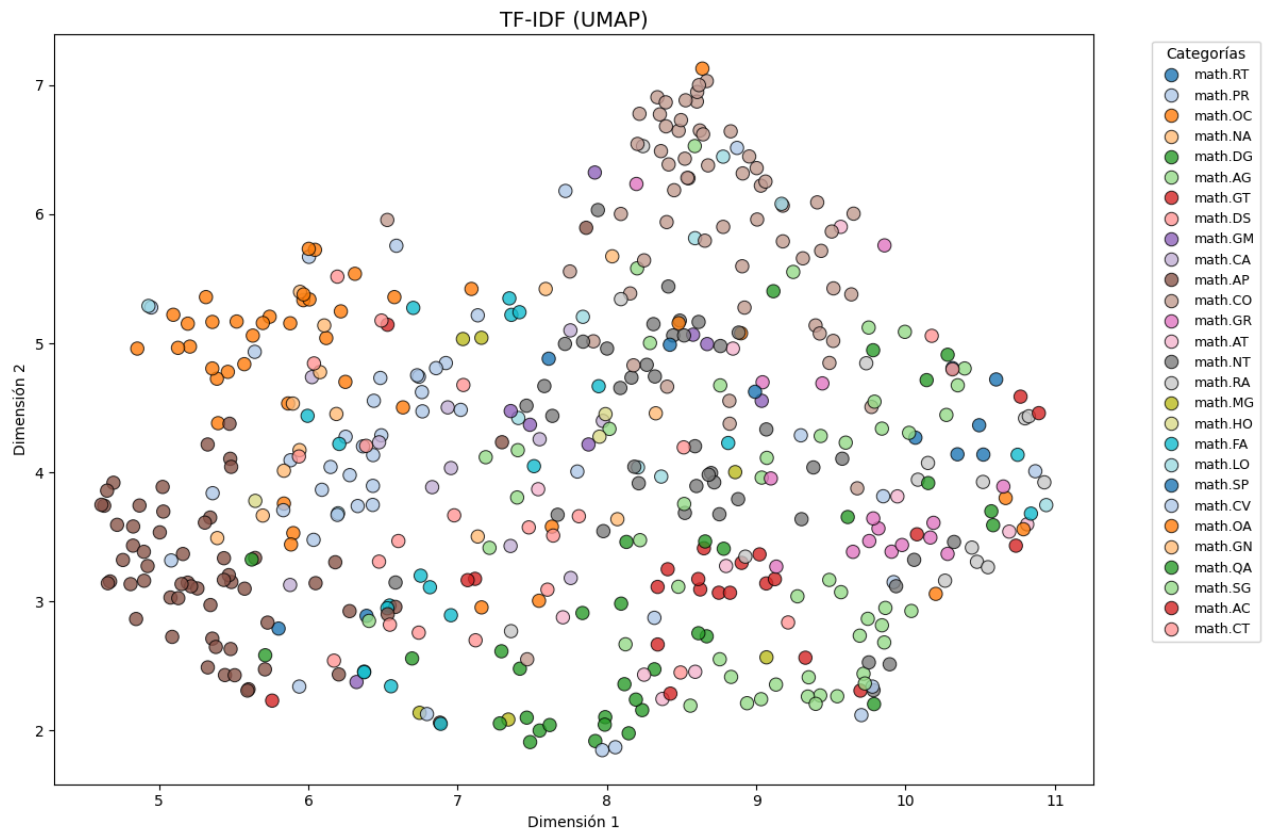
[Mostrar salida oculta](#)

- ✓ Podemos visualizar los embeddings de los papers con UMAP (cada punto es un paper)

```

visualizar_espacio_latente(
    X=X_test_tfidf,
    Y=y_test,
    clases=clases_matematicas,
    titulo="TF-IDF",
    n_samples=500,
    metodo='umap'
)

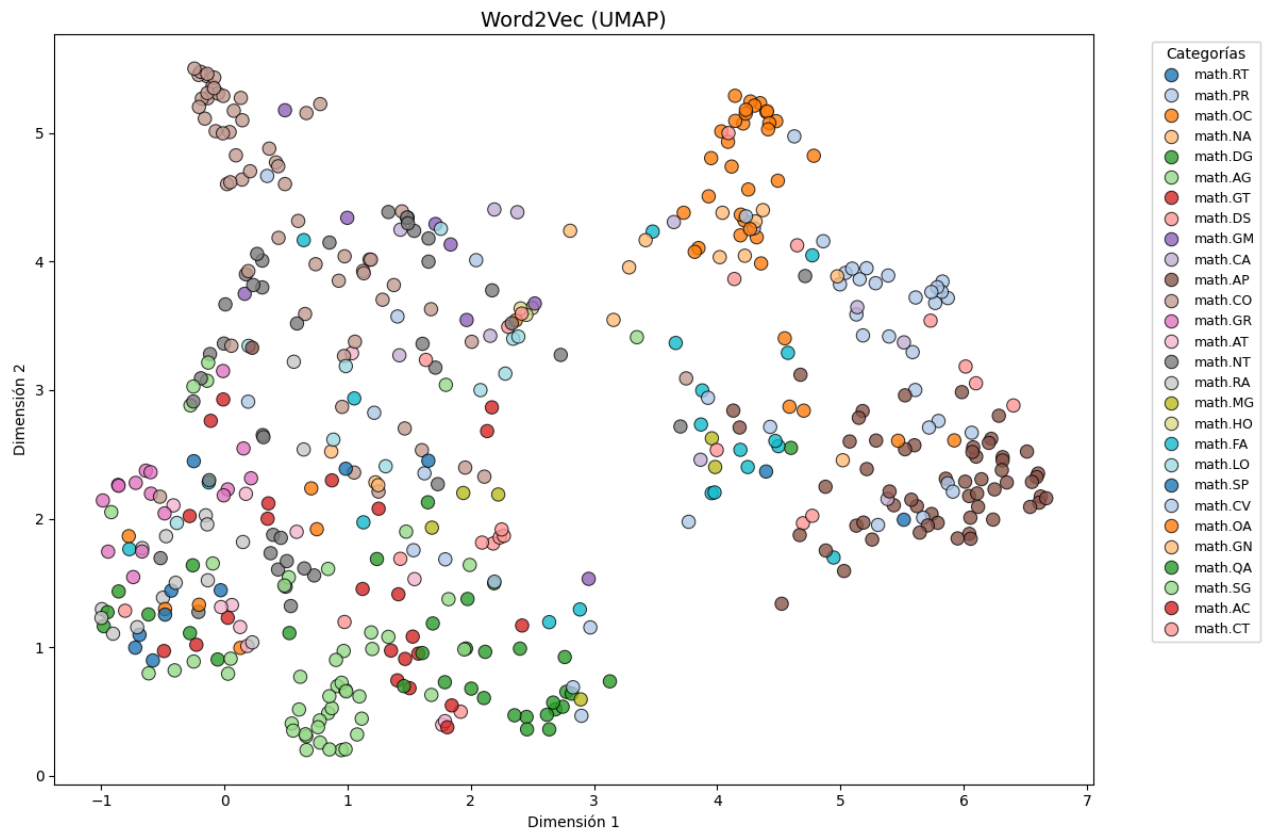
```



```

visualizar_espacio_latente(
    X=X_test_w2v,
    Y=y_test,
    clases=clases_matematicas,
    titulo="Word2Vec",
    n_samples=500,
    metodo='umap'
)

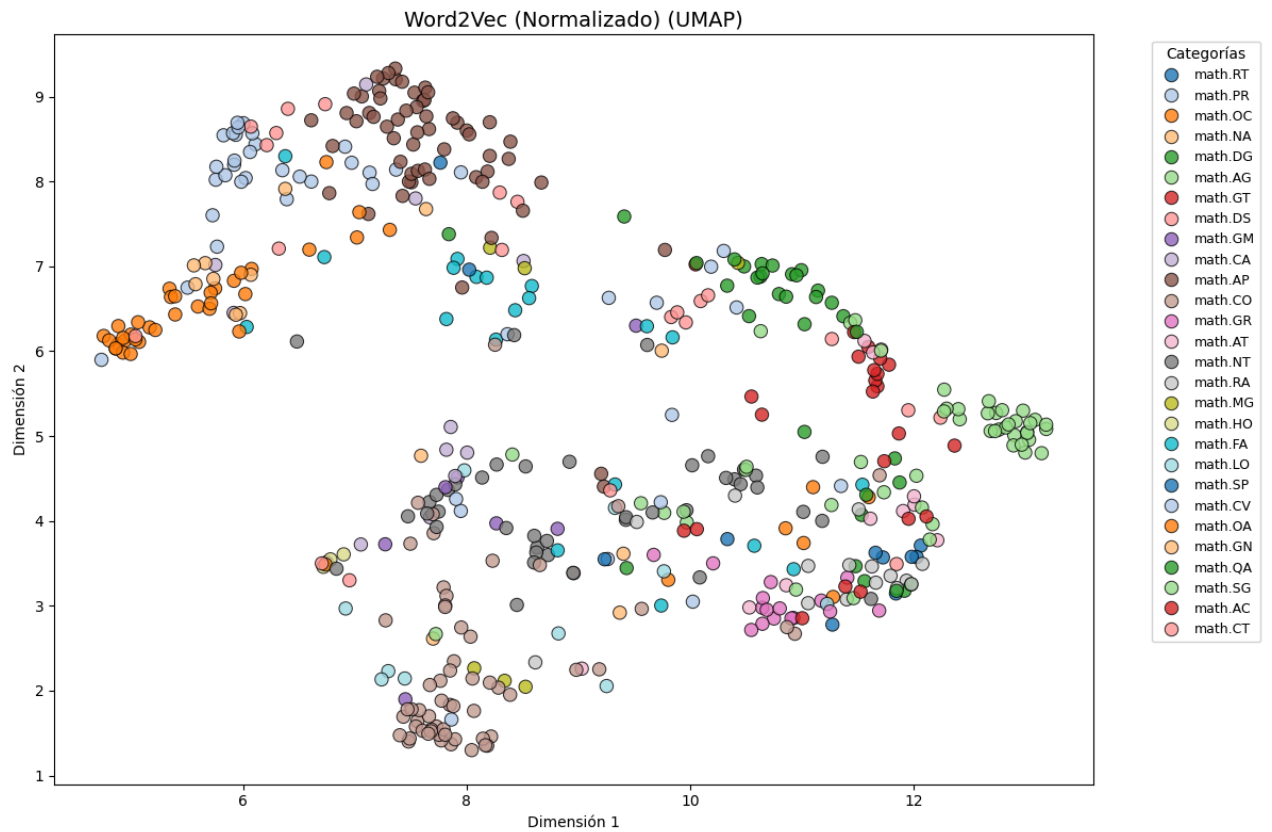
```



```

visualizar_espacio_latente(
    X=X_test_w2v_scaled,
    Y=y_test,
    clases=clases_matematicas,
    titulo="Word2Vec (Normalizado)",
    n_samples=500,
    metodo='umap'
)

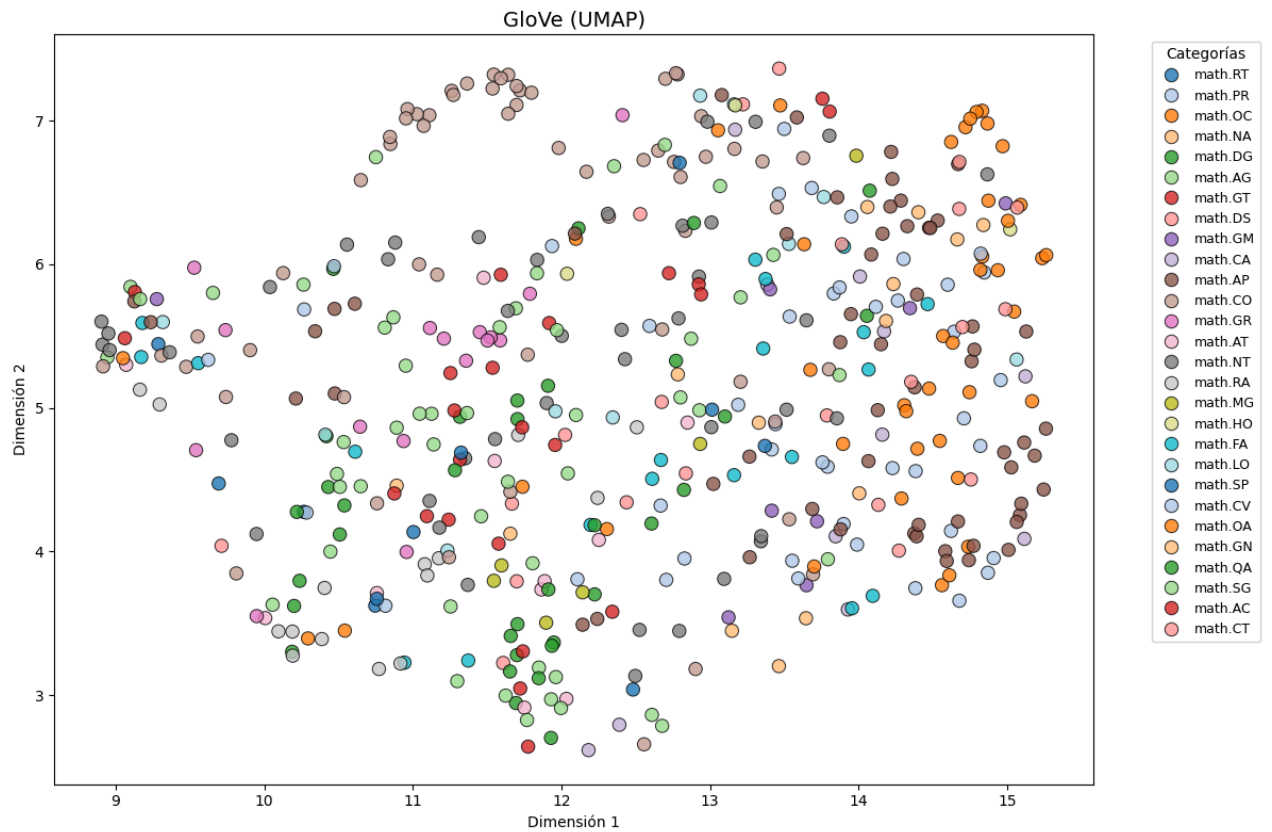
```



```

visualizar_espacio_latente(
    X=X_test_glove,
    Y=y_test,
    clases=clases_matematicas,
    titulo="GloVe",
    n_samples=500,
    metodo='umap'
)

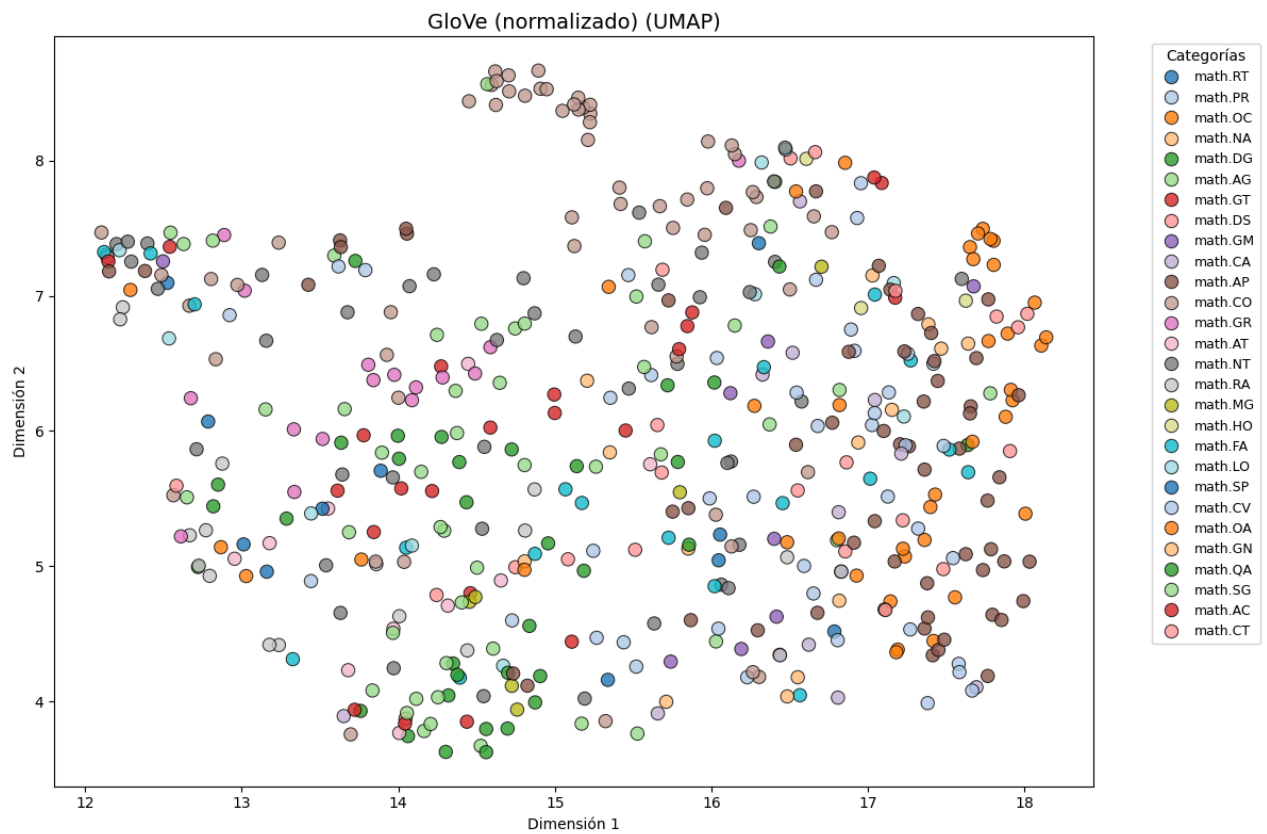
```



```

visualizar_espacio_latente(
    X=X_test_glove_scaled,
    Y=y_test,
    clases=clases_matematicas,
    titulo="GloVe (normalizado)",
    n_samples=500,
    metodo='umap'
)

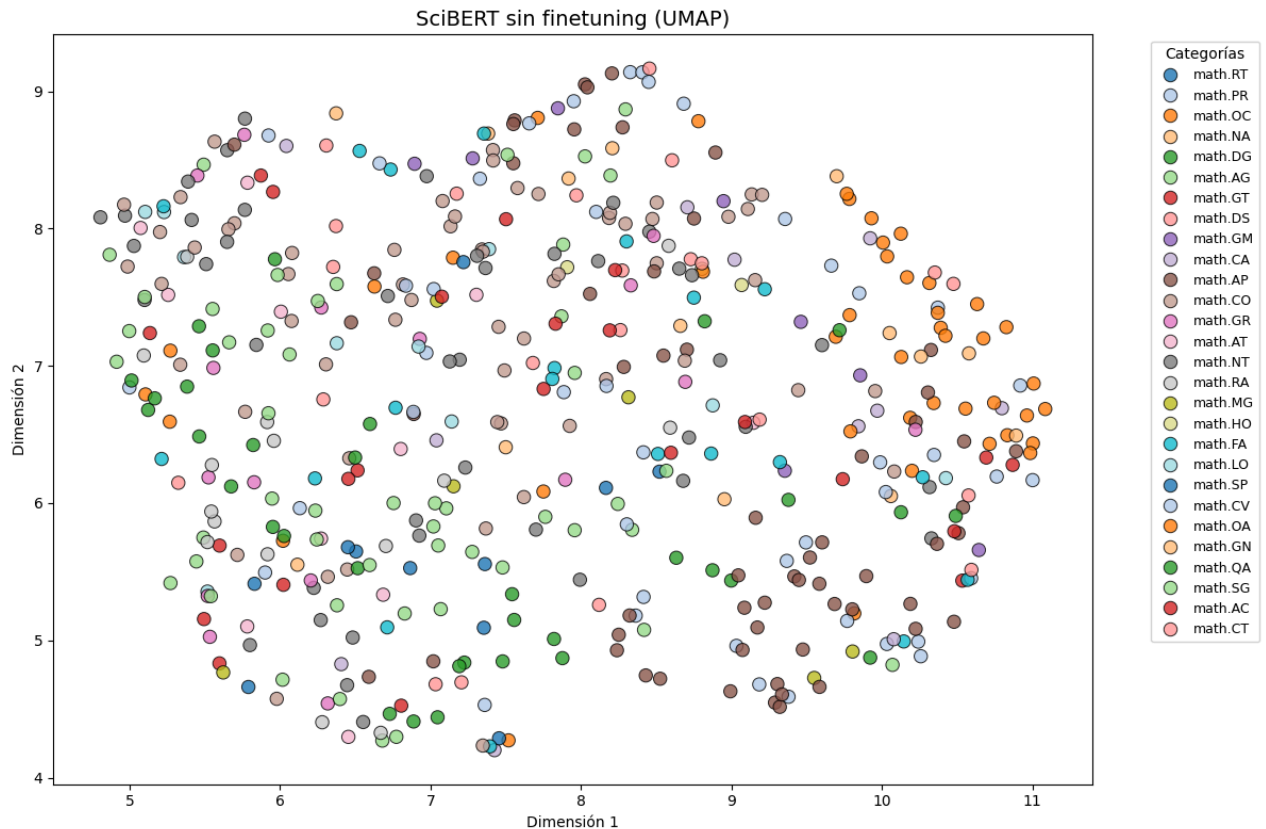
```



```

visualizar_espacio_latente(
    X=X_test_bert,
    Y=y_test,
    clases=clases_matematicas,
    titulo="SciBERT sin finetuning",
    n_samples=500,
    metodo='umap'
)

```

5. CLASIFICACIÓN

- Regresión Logística One vs Rest
- Perceptron multicapa
- Random Forest -> no da buenos resultados
- SVM lineal OvR
- SVM RBF

TF-IDF

```
print("--- MODELOS CON TF-IDF ---")

# Regresión Logística (OvR Wrapper)
ruta_logreg_tfidf = os.path.join(save_dir_base, 'logreg_tfidf.pkl')

if os.path.exists(ruta_logreg_tfidf):
    print("Modelo logístico OvR en Drive.")
    log_reg_tfidf = joblib.load(ruta_logreg_tfidf)
    t_mod = 25.68 # fue más porque le meti el array de tf-idf
    mem_mod = 1.07
else:
    print("Entrenando modelo logístico OvR.")
    t0_mod = time()
    mem0_mod = get_memory()

    base_lr = LogisticRegression(class_weight='balanced',
                                solver='lbfgs',
                                max_iter=1000,
                                random_state=42)

    log_reg_tfidf = OneVsRestClassifier(base_lr)
    log_reg_tfidf.fit(X_train_tfidf, y_train)
    joblib.dump(log_reg_tfidf, ruta_logreg_tfidf)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
```

```

print(f"Modelo logístico OvR guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_tfidf.predict(X_test_tfidf)
y_scores = log_reg_tfidf.predict_proba(X_test_tfidf)

total_time = round(t_emb_tfidf + t_mod, 2)
total_mem = round(mem_emb_tfidf + mem_mod, 2)
str_tiempo = f"{t_emb_tfidf} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_tfidf} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "TF-IDF", "Modelo": "RegLog (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# Perceptrón Multicapa (MLP)
ruta_mlp_tfidf = os.path.join(save_dir_base, 'mlp_tfidf.pkl')

if os.path.exists(ruta_mlp_tfidf):
    print("Modelo MLP en Drive.")
    mlp_tfidf = joblib.load(ruta_mlp_tfidf)
    t_mod = 37.53
    mem_mod = -738.69
else:
    print("Entrenando modelo MLP.")
    t0_mod = time()
    mem0_mod = get_memory()

    # MLP CON REGULARIZACIÓN
    mlp_tfidf = MLPClassifier(
        hidden_layer_sizes=(128, 64),
        activation='relu',
        alpha=0.01,          # Regularización L2
        early_stopping=True,  # Detiene el entrenamiento si el F1 no mejora
        max_iter=200,
        random_state=42
    )

    mlp_tfidf.fit(X_train_tfidf, y_train)
    joblib.dump(mlp_tfidf, ruta_mlp_tfidf)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo MLP guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# Evaluación
y_pred = mlp_tfidf.predict(X_test_tfidf)
y_scores = mlp_tfidf.predict_proba(X_test_tfidf)

total_time = round(t_emb_tfidf + t_mod, 2)
total_mem = round(mem_emb_tfidf + mem_mod, 2)
str_tiempo = f"{t_emb_tfidf} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_tfidf} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "TF-IDF",
    "Modelo": "MLP",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# SVM (OvR)
ruta_linearsvc_tfidf = os.path.join(save_dir_base, 'linearsvc_tfidf.pkl')

if os.path.exists(ruta_linearsvc_tfidf):
    print("Modelo LinearSVC (OvR) en Drive.")
    linearsvc_tfidf = joblib.load(ruta_linearsvc_tfidf)
    t_mod = 13.29
    mem_mod = 1.75

```

```

else:
    print("Entrenando modelo LinearSVC (OvR)...")
    t0_mod = time()
    mem0_mod = get_memory()

    # Configuramos el LinearSVC.
    base_svm = LinearSVC(
        class_weight='balanced',
        random_state=42,
        max_iter=1000
    )
    linearsvc_tfidf = OneVsRestClassifier(base_svm)

    # Entrenamos
    linearsvc_tfidf.fit(X_train_tfidf, y_train)

    joblib.dump(linearsvc_tfidf, ruta_linearsvc_tfidf)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo LinearSVC (OvR) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# Evaluación
y_pred = linearsvc_tfidf.predict(X_test_tfidf)
y_scores = linearsvc_tfidf.decision_function(X_test_tfidf)

total_time = round(t_emb_tfidf + t_mod, 2)
total_mem = round(mem_emb_tfidf + mem_mod, 2)
str_tiempo = f"{t_emb_tfidf} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_tfidf} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "TF-IDF",
    "Modelo": "LinearSVC (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# SVM RBF (OvR) - APROXIMADO CON NYSTROEM
ruta_svc_rbf_tfidf = os.path.join(save_dir_base, 'svc_rbf_tfidf.pkl')

if os.path.exists(ruta_svc_rbf_tfidf):
    print("Modelo SVC (RBF Aproximado) en Drive.")
    svc_rbf_tfidf = joblib.load(ruta_svc_rbf_tfidf)
    t_mod = 0
    mem_mod = 0
else:
    print("Entrenando modelo SVC (RBF Aproximado con Nyström) OvR... ")
    gc.collect()

    t0_mod = time()
    mem0_mod = get_memory()

    # Creamos la aproximación del Kernel RBF
    # n_components es cuántos "puntos de anclaje" usa. 500 es un buen equilibrio rapidez/precisión.
    nystroem_rbf = Nystroem(kernel='rbf', gamma=0.1, n_components=500, random_state=42)

    # Usamos el modelo lineal (que es más rápido)
    base_svm_fast = LinearSVC(class_weight='balanced', max_iter=1000, random_state=42)

    # Los unimos en un pipeline
    modelo_base_aproximado = make_pipeline(nystroem_rbf, base_svm_fast)

    # Lo envolvemos en el clasificador multietiqueta
    svc_rbf_tfidf = OneVsRestClassifier(modelo_base_aproximado)

    # Entrenamos
    svc_rbf_tfidf.fit(X_train_tfidf, y_train)

    joblib.dump(svc_rbf_tfidf, ruta_svc_rbf_tfidf)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo SVC (RBF Aproximado) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# Evaluación
y_pred = svc_rbf_tfidf.predict(X_test_tfidf)
y_scores = svc_rbf_tfidf.decision_function(X_test_tfidf)

```

```

# Tiempos y memorias totales
total_time = round(t_emb_tfidf + t_mod, 2)
total_mem = round(mem_emb_tfidf + mem_mod, 2)
str_tiempo = f"{t_emb_tfidf} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_tfidf} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "TF-IDF",
    "Modelo": "SVC (RBF Aproximado OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# RANDOM FOREST
ruta_rf_tfidf = os.path.join(save_dir_base, 'rf_tfidf.pkl')

if os.path.exists(ruta_rf_tfidf):
    print("Modelo Random Forest (OvR) en Drive.")
    rf_tfidf = joblib.load(ruta_rf_tfidf)
    t_mod = 20.6
    mem_mod = 20541.6
else:
    print("Entrenando modelo Random Forest (OvR)...")
    # Forzamos limpieza de RAM
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    # Configuramos el Random Forest base
    rf_tfidf = RandomForestClassifier(
        n_estimators=100,          # 100 árboles
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    )

    rf_tfidf.fit(X_train_tfidf, y_train)

    joblib.dump(rf_tfidf, ruta_rf_tfidf)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo Random Forest guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# Evaluación
y_pred = rf_tfidf.predict(X_test_tfidf)
probs_lista = rf_tfidf.predict_proba(X_test_tfidf)

# Extraemos solo la probabilidad de la clase 1 para cada categoría y transponemos
y_scores = np.array([clase[:, 1] for clase in probs_lista]).T

total_time = round(t_emb_tfidf + t_mod, 2)
total_mem = round(mem_emb_tfidf + mem_mod, 2)
str_tiempo = f"{t_emb_tfidf} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_tfidf} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "TF-IDF",
    "Modelo": "Random Forest (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

del X_train_tfidf, X_test_tfidf
gc.collect()

--- MODELOS CON TF-IDF ---
Modelo logístico OvR en Drive.
Modelo MLP en Drive.
Modelo LinearSVC (OvR) en Drive.
Modelo SVC (RBF Aproximado) en Drive.

```

Modelo Random Forest (OvR) en Drive.
146036

El entrenamiento de un SVC con kernel RBF tradicional resultaba intratable computacionalmente debido a su complejidad cuadrática $O(N^2)$ al combinarse con un enfoque OvR para 29 clases. Para solucionarlo preservando la no linealidad, implementé una aproximación del kernel mediante el método de Nyström proyectado sobre un clasificador LinearSVC

✓ Clasificación con Word2Vec

```
print("--- MODELOS CON WORD2VEC ---")

# Regresión Logística (OvR Wrapper)
ruta_logreg_w2v = os.path.join(save_dir_base, 'logreg_w2v.pkl')

if os.path.exists(ruta_logreg_w2v):
    print("Modelo logístico OvR en Drive.")
    log_reg_w2v = joblib.load(ruta_logreg_w2v)
    t_mod = 2.2
    mem_mod = 26.62
else:
    print("Entrenando modelo logístico OvR.")
    # Forzamos limpieza de RAM
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_lr = LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
    log_reg_w2v = OneVsRestClassifier(base_lr)
    log_reg_w2v.fit(X_train_w2v, y_train)
    joblib.dump(log_reg_w2v, ruta_logreg_w2v)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo logístico OvR guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_w2v.predict(X_test_w2v)
y_scores = log_reg_w2v.predict_proba(X_test_w2v)

total_time = round(t_emb_w2v + t_mod, 2)
total_mem = round(mem_emb_w2v + mem_mod, 2)
str_tiempo = f"{t_emb_w2v} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_w2v} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "Word2Vec", "Modelo": "RegLog (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# RANDOM FOREST
ruta_rf_w2v = os.path.join(save_dir_base, 'rf_w2v.pkl')

if os.path.exists(ruta_rf_w2v):
    print("Modelo Random Forest en Drive.")
    rf_w2v = joblib.load(ruta_rf_w2v)
    t_mod = 15.89
    mem_mod = 14223.65
else:
    print("Entrenando modelo Random Forest...")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    rf_w2v = RandomForestClassifier(
        n_estimators=100,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    )

    rf_w2v.fit(X_train_w2v, y_train)

    joblib.dump(rf_w2v, ruta_rf_w2v)
```

```

t_mod = round((time() - t0_mod) / 60, 2)
mem_mod = round(get_memory() - mem0_mod, 2)
print(f"Modelo Random Forest guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = rf_w2v.predict(X_test_w2v)
probs_lista = rf_w2v.predict_proba(X_test_w2v)

# Extraemos solo la probabilidad de la clase 1 para cada categoría y transponemos
y_scores = np.array([clase[:, 1] for clase in probs_lista]).T

total_time = round(t_emb_w2v + t_mod, 2)
total_mem = round(mem_emb_w2v + mem_mod, 2)
str_tiempo = f"{t_emb_w2v} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_w2v} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "Word2Vec",
    "Modelo": "Random Forest",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

print("\n--- MODELOS CON WORD2VEC (Norm) ---")

# Regresión Logística (OvR Wrapper) Norm
ruta_logreg_w2v_scaled = os.path.join(save_dir_base, 'logreg_w2v_scaled.pkl')

if os.path.exists(ruta_logreg_w2v_scaled):
    print("Modelo logístico OvR Norm en Drive.")
    log_reg_w2v_scaled = joblib.load(ruta_logreg_w2v_scaled)
    t_mod = 1.51
    mem_mod = 11.28
else:
    print("Entrenando modelo logístico OvR Norm.")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_lr_scaled = LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
    log_reg_w2v_scaled = OneVsRestClassifier(base_lr_scaled)
    log_reg_w2v_scaled.fit(X_train_w2v_scaled, y_train)
    joblib.dump(log_reg_w2v_scaled, ruta_logreg_w2v_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo logístico OvR Norm guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_w2v_scaled.predict(X_test_w2v_scaled)
y_scores = log_reg_w2v_scaled.predict_proba(X_test_w2v_scaled)

total_time = round(t_emb_w2v + t_mod, 2)
total_mem = round(mem_emb_w2v + mem_mod, 2)
str_tiempo = f"{t_emb_w2v} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_w2v} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "Word2Vec (Norm)", "Modelo": "RegLog (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# Perceptrón Multicapa (MLP) Norm
ruta_mlp_w2v_scaled = os.path.join(save_dir_base, 'mlp_w2v_scaled.pkl')

if os.path.exists(ruta_mlp_w2v_scaled):
    print("Modelo MLP Norm en Drive.")
    mlp_w2v_scaled = joblib.load(ruta_mlp_w2v_scaled)
    t_mod = 12.89
    mem_mod = 1.15
else:
    print("Entrenando modelo MLP Norm.")

```

```

gc.collect()
t0_mod = time()
mem0_mod = get_memory()

mlp_w2v_scaled = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation='relu',
    alpha=0.01,
    early_stopping=True,
    max_iter=200,
    random_state=42
)
mlp_w2v_scaled.fit(X_train_w2v_scaled, y_train)
joblib.dump(mlp_w2v_scaled, ruta_mlp_w2v_scaled)

t_mod = round((time() - t0_mod) / 60, 2)
mem_mod = round(get_memory() - mem0_mod, 2)
print(f"Modelo MLP Norm guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = mlp_w2v_scaled.predict(X_test_w2v_scaled)
y_scores = mlp_w2v_scaled.predict_proba(X_test_w2v_scaled)

total_time = round(t_emb_w2v + t_mod, 2)
total_mem = round(mem_emb_w2v + mem_mod, 2)
str_tiempo = f"{t_emb_w2v} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_w2v} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "Word2Vec (Norm)", "Modelo": "MLP",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# LinearSVC (OvR) - NORMALIZADO
ruta_linearsvc_w2v_scaled = os.path.join(save_dir_base, 'linearsvc_w2v_scaled.pkl')

if os.path.exists(ruta_linearsvc_w2v_scaled):
    print("Modelo LinearSVC (OvR) W2V en Drive.")
    linearsvc_w2v_scaled = joblib.load(ruta_linearsvc_w2v_scaled)
    t_mod = 14.19
    mem_mod = 0.32
else:
    print("Entrenando modelo LinearSVC (OvR) Normalizado.")
    # Forzamos limpieza de RAM
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_svm = LinearSVC(
        class_weight='balanced',
        random_state=42,
        max_iter=1000
    )
    linearsvc_w2v_scaled = OneVsRestClassifier(base_svm)
    linearsvc_w2v_scaled.fit(X_train_w2v_scaled, y_train)
    joblib.dump(linearsvc_w2v_scaled, ruta_linearsvc_w2v_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo LinearSVC (OvR) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = linearsvc_w2v_scaled.predict(X_test_w2v_scaled)
y_scores = linearsvc_w2v_scaled.decision_function(X_test_w2v_scaled)

total_time = round(t_emb_w2v + t_mod, 2)
total_mem = round(mem_emb_w2v + mem_mod, 2)
str_tiempo = f"{t_emb_w2v} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_w2v} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "Word2Vec", "Modelo": "LinearSVC (Norm)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro', zero_division=0), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro', zero_division=0), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

```

```

}))

# SVM RBF (OvR) - APROXIMADO CON NYSTROEM - NORMALIZADO
ruta_svc_rbf_w2v_scaled = os.path.join(save_dir_base, 'svc_rbf_w2v_scaled.pkl')

if os.path.exists(ruta_svc_rbf_w2v_scaled):
    print("Modelo SVC (RBF Aproximado) en Drive.")
    svc_rbf_w2v = joblib.load(ruta_svc_rbf_w2v_scaled)
    t_mod = 44.7
    mem_mod = 526.26
else:
    print("Entrenando modelo SVC (RBF Aproximado con Nyström) OvR... ")
    # Forzamos limpieza de RAM
    gc.collect()

    t0_mod = time()
    mem0_mod = get_memory()

    # Aproximación de Nyström para el kernel RBF
    nystroem_rbf = Nystroem(kernel='rbf', gamma=0.1, n_components=500, random_state=42)

    # Modelo lineal rápido
    base_svm_fast = LinearSVC(class_weight='balanced', max_iter=1000, random_state=42)

    # Creamos el pipeline y lo envolvemos para multietiqueta
    modelo_base_aproximado = make_pipeline(nystroem_rbf, base_svm_fast)
    svc_rbf_w2v = OneVsRestClassifier(modelo_base_aproximado)

    # Entrenamos
    svc_rbf_w2v.fit(X_train_w2v_scaled, y_train)

    joblib.dump(svc_rbf_w2v, ruta_svc_rbf_w2v_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo SVC (RBF Aproximado) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = svc_rbf_w2v.predict(X_test_w2v_scaled)
y_scores = svc_rbf_w2v.decision_function(X_test_w2v_scaled)

total_time = round(t_emb_w2v + t_mod, 2)
total_mem = round(mem_emb_w2v + mem_mod, 2)
str_tiempo = f"{t_emb_w2v} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_w2v} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "Word2Vec",
    "Modelo": "SVC (RBF Aproximado - Norm)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

del X_train_w2v, X_test_w2v, X_train_w2v_scaled, X_test_w2v_scaled
gc.collect()

```

```

--- MODELOS CON WORD2VEC ---
Modelo logístico OvR en Drive.
Modelo LinearSVC (OvR) W2V en Drive.
Modelo SVC (RBF Aproximado) en Drive.
Modelo Random Forest en Drive.

--- MODELOS CON WORD2VEC (Norm) ---
Modelo logístico OvR Norm en Drive.
Modelo MLP Norm en Drive.
56198

```

✓ Clasificación con GloVe

```

print("--- MODELOS CON GLOVE ---")

# Regresión Logística (OvR Wrapper)
ruta_logreg_glove = os.path.join(save_dir_base, 'logreg_glove.pkl')

if os.path.exists(ruta_logreg_glove):
    print("Modelo logístico OvR en Drive.")
    log_reg_glove = joblib.load(ruta_logreg_glove)
    t_mod = 1.71

```



```

        mem_mod = 285.4
    else:
        print("Entrenando modelo logístico OvR.")
        gc.collect()
        t0_mod = time()
        mem0_mod = get_memory()

        base_lr = LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
        log_reg_glove = OneVsRestClassifier(base_lr)
        log_reg_glove.fit(X_train_glove, y_train)
        joblib.dump(log_reg_glove, ruta_logreg_glove)

        t_mod = round((time() - t0_mod) / 60, 2)
        mem_mod = round(get_memory() - mem0_mod, 2)
        print(f"Modelo logístico OvR guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_glove.predict(X_test_glove)
y_scores = log_reg_glove.predict_proba(X_test_glove)

total_time = round(t_emb_glove + t_mod, 2)
total_mem = round(mem_emb_glove + mem_mod, 2)
str_tiempo = f"{t_emb_glove} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_glove} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "GloVe", "Modelo": "RegLog (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# RANDOM FOREST
ruta_rf_glove = os.path.join(save_dir_base, 'rf_glove.pkl')

if os.path.exists(ruta_rf_glove):
    print("Modelo Random Forest en Drive.")
    rf_glove = joblib.load(ruta_rf_glove)
    t_mod = 10.91
    mem_mod = 17476.14
else:
    print("Entrenando modelo Random Forest...")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    rf_glove = RandomForestClassifier(
        n_estimators=100,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    )

    rf_glove.fit(X_train_glove, y_train)

    joblib.dump(rf_glove, ruta_rf_glove)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo Random Forest guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = rf_glove.predict(X_test_glove)
probs_lista = rf_glove.predict_proba(X_test_glove)

y_scores = np.array([clase[:, 1] for clase in probs_lista]).T

total_time = round(t_emb_glove + t_mod, 2)
total_mem = round(mem_emb_glove + mem_mod, 2)
str_tiempo = f"{t_emb_glove} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_glove} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "GloVe",
    "Modelo": "Random Forest",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,

```

```

    memoria (MB) : str_memoria
))

print("\n--- MODELOS CON GLOVE (Norm) ---")

# Regresión Logística (OvR Wrapper) Norm
ruta_logreg_glove_scaled = os.path.join(save_dir_base, 'logreg_glove_scaled.pkl')

if os.path.exists(ruta_logreg_glove_scaled):
    print("Modelo logístico OvR Norm en Drive.")
    log_reg_glove_scaled = joblib.load(ruta_logreg_glove_scaled)
    t_mod = 1.64
    mem_mod = 279.12
else:
    print("Entrenando modelo logístico OvR Norm.")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_lr_scaled = LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
    log_reg_glove_scaled = OneVsRestClassifier(base_lr_scaled)
    log_reg_glove_scaled.fit(X_train_glove_scaled, y_train)
    joblib.dump(log_reg_glove_scaled, ruta_logreg_glove_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo logístico OvR Norm guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_glove_scaled.predict(X_test_glove_scaled)
y_scores = log_reg_glove_scaled.predict_proba(X_test_glove_scaled)

total_time = round(t_emb_glove + t_mod, 2)
total_mem = round(mem_emb_glove + mem_mod, 2)
str_tiempo = f"{t_emb_glove} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_glove} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "GloVe (Norm)", "Modelo": "RegLog (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# Perceptrón Multicapa (MLP) Norm
ruta_mlp_glove_scaled = os.path.join(save_dir_base, 'mlp_glove_scaled.pkl')

if os.path.exists(ruta_mlp_glove_scaled):
    print("Modelo MLP Norm en Drive.")
    mlp_glove_scaled = joblib.load(ruta_mlp_glove_scaled)
    t_mod = 9.93
    mem_mod = -4.19
else:
    print("Entrenando modelo MLP Norm.")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    mlp_glove_scaled = MLPClassifier(
        hidden_layer_sizes=(128, 64),
        activation='relu',
        alpha=0.01,
        early_stopping=True,
        max_iter=200,
        random_state=42
    )

    mlp_glove_scaled.fit(X_train_glove_scaled, y_train)
    joblib.dump(mlp_glove_scaled, ruta_mlp_glove_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo MLP Norm guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = mlp_glove_scaled.predict(X_test_glove_scaled)
y_scores = mlp_glove_scaled.predict_proba(X_test_glove_scaled)

total_time = round(t_emb_glove + t_mod, 2)

```

```

total_mem = round(mem_emb_glove + mem_mod, 2)
str_tiempo = f"{t_emb_glove} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_glove} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "GloVe (Norm)", "Modelo": "MLP",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# LinearSVC (OvR) - NORMALIZADO
ruta_linearsvc_glove_scaled = os.path.join(save_dir_base, 'linearsvc_glove_scaled.pkl')

if os.path.exists(ruta_linearsvc_glove_scaled):
    print("Modelo LinearSVC (OvR) GloVe en Drive.")
    linearsvc_glove = joblib.load(ruta_linearsvc_glove_scaled)
    t_mod = 5.53
    mem_mod = 2.25
else:
    print("Entrenando modelo LinearSVC (OvR).")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_svm = LinearSVC(
        class_weight='balanced',
        random_state=42,
        max_iter=1000
    )

    linearsvc_glove = OneVsRestClassifier(base_svm)
    linearsvc_glove.fit(X_train_glove_scaled, y_train)
    joblib.dump(linearsvc_glove, ruta_linearsvc_glove_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo LinearSVC (OvR) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = linearsvc_glove.predict(X_test_glove_scaled)
y_scores = linearsvc_glove.decision_function(X_test_glove_scaled)

total_time = round(t_emb_glove + t_mod, 2)
total_mem = round(mem_emb_glove + mem_mod, 2)
str_tiempo = f"{t_emb_glove} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_glove} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "GloVe", "Modelo": "LinearSVC (Norm)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# SVM RBF (OvR) - APROXIMADO CON NYSTROEM - NORMALIZADO
ruta_svc_rbf_glove_scaled = os.path.join(save_dir_base, 'svc_rbf_glove_scaled.pkl')

if os.path.exists(ruta_svc_rbf_glove_scaled):
    print("Modelo SVC (RBF Aproximado) en Drive.")
    svc_rbf_glove = joblib.load(ruta_svc_rbf_glove_scaled)
    t_mod = 51.78
    mem_mod = 63.75
else:
    print("Entrenando modelo SVC (RBF Aproximado con Nyström) OvR... ")
    gc.collect()

    t0_mod = time()
    mem0_mod = get_memory()

    # Aproximación de Nyström para el kernel RBF
    nystroem_rbf = Nystroem(kernel='rbf', gamma=0.1, n_components=500, random_state=42)

    # Modelo lineal rápido
    base_svm_fast = LinearSVC(class_weight='balanced', max_iter=1000, random_state=42)

    # Creamos el pipeline y lo envolvemos para multietiqueta

```

```

modelo_base_aproximado = make_pipeline(nystroem_rbf, base_svm_fast)
svc_rbf_glove = OneVsRestClassifier(modelo_base_aproximado)

# Entrenamos
svc_rbf_glove.fit(X_train_glove_scaled, y_train)

joblib.dump(svc_rbf_glove, ruta_svc_rbf_glove_scaled)

t_mod = round((time() - t0_mod) / 60, 2)
mem_mod = round(get_memory() - mem0_mod, 2)
print(f"Modelo SVC (RBF Aproximado) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = svc_rbf_glove.predict(X_test_glove_scaled)
y_scores = svc_rbf_glove.decision_function(X_test_glove_scaled)

total_time = round(t_emb_glove + t_mod, 2)
total_mem = round(mem_emb_glove + mem_mod, 2)
str_tiempo = f"{t_emb_glove} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_glove} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "GloVe",
    "Modelo": "SVC (RBF Aproximado - Norm)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

del X_train_glove_scaled, X_test_glove_scaled, X_train_glove, X_test_glove
gc.collect()

```

```

--- MODELOS CON GLOVE ---
Modelo logístico OvR en Drive.
Modelo LinearSVC (OvR) GloVe en Drive.
Modelo SVC (RBF Aproximado) en Drive.
Modelo Random Forest en Drive.

```

```

--- MODELOS CON GLOVE (Norm) ---
Modelo logístico OvR Norm en Drive.
Modelo MLP Norm en Drive.
48

```

✓ Clasificación con SciBert sin ajuste fino

```

print("--- MODELOS CON SCIBERT---")

# Regresión Logística (OvR Wrapper)
ruta_logreg_bert = os.path.join(save_dir_base, 'logreg_bert.pkl')

if os.path.exists(ruta_logreg_bert):
    print("Modelo logístico OvR en Drive.")
    log_reg_bert = joblib.load(ruta_logreg_bert)
    t_mod = 20
    mem_mod = -46.11
else:
    print("Entrenando modelo logístico OvR.")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_lr = LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
    log_reg_bert = OneVsRestClassifier(base_lr)
    log_reg_bert.fit(X_train_bert, y_train)
    joblib.dump(log_reg_bert, ruta_logreg_bert)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo logístico OvR guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_bert.predict(X_test_bert)
y_scores = log_reg_bert.predict_proba(X_test_bert)

total_time = round(t_emb_bert + t_mod, 2)
total_mem = round(mem_emb_bert + mem_mod, 2)
str_tiempo = f"{t_emb_bert} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_bert} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "SciBERT", "Modelo": "Logístico (OvR)"
})

```

```

Embedding = SciBERT, Modelo = RegLog (OvR),
"F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
"F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
"Accuracy": round(accuracy_score(y_test, y_pred), 4),
"PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
"Tiempo (min)": str_tiempo,
"Memoria (MB)": str_memoria
})

# RANDOM FOREST
ruta_rf_bert = os.path.join(save_dir_base, 'rf_bert.pkl')

if os.path.exists(ruta_rf_bert):
    print("Modelo Random Forest en Drive.")
    rf_bert = joblib.load(ruta_rf_bert)
    t_mod = 35.86
    mem_mod = 18555.28
else:
    print("Entrenando modelo Random Forest...")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    rf_bert = RandomForestClassifier(
        n_estimators=100,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    )

    rf_bert.fit(X_train_bert, y_train)

    joblib.dump(rf_bert, ruta_rf_bert)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo Random Forest guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = rf_bert.predict(X_test_bert)
probs_lista = rf_bert.predict_proba(X_test_bert)

y_scores = np.array([clase[:, 1] for clase in probs_lista]).T

total_time = round(t_emb_bert + t_mod, 2)
total_mem = round(mem_emb_bert + mem_mod, 2)
str_tiempo = f"{t_emb_bert} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_bert} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "SciBERT",
    "Modelo": "Random Forest",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

print("\n--- MODELOS CON SCIBERT (Normalizado) ---")

# Regresión Logística (OvR) NORM
ruta_logreg_bert_scaled = os.path.join(save_dir_base, 'logreg_bert_scaled.pkl')

if os.path.exists(ruta_logreg_bert_scaled):
    print("Modelo logístico OvR Norm en Drive.")
    log_reg_bert_scaled = joblib.load(ruta_logreg_bert_scaled)
    t_mod = 3.75
    mem_mod = 47.13
else:
    print("Entrenando modelo logístico OvR Norm.")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    base_lr_scaled = LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
    log_reg_bert_scaled = OneVsRestClassifier(base_lr_scaled)
    log_reg_bert_scaled.fit(X_train_bert_scaled, y_train)
    joblib.dump(log_reg_bert_scaled, ruta_logreg_bert_scaled)

```

```

t_mod = round((time() - t0_mod) / 60, 2)
mem_mod = round(get_memory() - mem0_mod, 2)
print(f"Modelo logístico OvR Norm guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = log_reg_bert_scaled.predict(X_test_bert_scaled)
y_scores = log_reg_bert_scaled.predict_proba(X_test_bert_scaled)

total_time = round(t_emb_bert + t_mod, 2)
total_mem = round(mem_emb_bert + mem_mod, 2)
str_tiempo = f"{t_emb_bert} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_bert} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "SciBERT (Norm)", "Modelo": "RegLog (OvR)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# Perceptrón Multicapa (MLP) NORMALIZADO
ruta_mlp_bert_scaled = os.path.join(save_dir_base, 'mlp_bert_scaled.pkl')

if os.path.exists(ruta_mlp_bert_scaled):
    print("Modelo MLP Norm en Drive.")
    mlp_bert_scaled = joblib.load(ruta_mlp_bert_scaled)
    t_mod = 69.62
    mem_mod = 0.66
else:
    print("Entrenando modelo MLP Norm.")
    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    mlp_bert_scaled = MLPClassifier(
        hidden_layer_sizes=(256, 64),
        activation='relu',
        alpha=0.01,
        early_stopping=True,
        max_iter=200,
        random_state=42
    )

    mlp_bert_scaled.fit(X_train_bert_scaled, y_train)
    joblib.dump(mlp_bert_scaled, ruta_mlp_bert_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo MLP Norm guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = mlp_bert_scaled.predict(X_test_bert_scaled)
y_scores = mlp_bert_scaled.predict_proba(X_test_bert_scaled)

total_time = round(t_emb_bert + t_mod, 2)
total_mem = round(mem_emb_bert + mem_mod, 2)
str_tiempo = f"{t_emb_bert} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_bert} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "SciBERT (Norm)", "Modelo": "MLP",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# LinearSVC (OvR) - NORMALIZADO
ruta_linearsvc_bert_scaled = os.path.join(save_dir_base, 'linearsvc_bert_scaled.pkl')

if os.path.exists(ruta_linearsvc_bert_scaled):
    print("Modelo LinearSVC (OvR) en Drive.")
    linearsvc_bert = joblib.load(ruta_linearsvc_bert_scaled)
    t_mod = 32.05
    mem_mod = 15.96
else:
    print("Entrenando modelo LinearSVC (OvR).")

```

```

gc.collect()
t0_mod = time()
mem0_mod = get_memory()

base_svm = LinearSVC(
    C=0.01,
    class_weight='balanced',
    random_state=42,
    max_iter=1000
)

linearsvc_bert = OneVsRestClassifier(base_svm)
linearsvc_bert.fit(X_train_bert_scaled, y_train)
joblib.dump(linearsvc_bert, ruta_linearsvc_bert_scaled)

t_mod = round((time() - t0_mod) / 60, 2)
mem_mod = round(get_memory() - mem0_mod, 2)
print(f"Modelo LinearSVC (OvR) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = linearsvc_bert.predict(X_test_bert_scaled)
y_scores = linearsvc_bert.decision_function(X_test_bert_scaled)

total_time = round(t_emb_bert + t_mod, 2)
total_mem = round(mem_emb_bert + mem_mod, 2)
str_tiempo = f"{t_emb_bert} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_bert} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "SciBERT", "Modelo": "LinearSVC (Norm)",
    "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred), 4),
    "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
    "Tiempo (min)": str_tiempo,
    "Memoria (MB)": str_memoria
})

# SVM RBF (OvR) - APROXIMADO CON NYSTROEM
ruta_svc_rbf_bert_scaled = os.path.join(save_dir_base, 'svc_rbf_bert_scaled.pkl')

if os.path.exists(ruta_svc_rbf_bert_scaled):
    print("Modelo SVC (RBF Aproximado) en Drive.")
    svc_rbf_bert = joblib.load(ruta_svc_rbf_bert_scaled)
    t_mod = 47.39
    mem_mod = 150.27
else:
    print("Entrenando modelo SVC (RBF Aproximado con Nyström) OvR... ")
    gc.collect()

    t0_mod = time()
    mem0_mod = get_memory()

    # Aproximación de Nyström para el kernel RBF
    nystroem_rbf = Nystroem(kernel='rbf', gamma=0.1, n_components=500, random_state=42)

    # Modelo lineal rápido
    base_svm_fast = LinearSVC(class_weight='balanced', max_iter=1000, random_state=42)

    # Creamos el pipeline y lo envolvemos para multietiqueta
    modelo_base_aproximado = make_pipeline(nystroem_rbf, base_svm_fast)
    svc_rbf_bert = OneVsRestClassifier(modelo_base_aproximado)

    # Entrenamos
    svc_rbf_bert.fit(X_train_bert_scaled, y_train)

    joblib.dump(svc_rbf_bert, ruta_svc_rbf_bert_scaled)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo SVC (RBF Aproximado) guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

y_pred = svc_rbf_bert.predict(X_test_bert_scaled)
y_scores = svc_rbf_bert.decision_function(X_test_bert_scaled)

total_time = round(t_emb_bert + t_mod, 2)
total_mem = round(mem_emb_bert + mem_mod, 2)
str_tiempo = f"{t_emb_bert} + {t_mod} = {total_time}"
str_memoria = f"{mem_emb_bert} + {mem_mod} = {total_mem}"

resultados_globales.append({
    "Embedding": "SciBERT",
    "Modelo": "SVC (RBF Aproximado con Nyström)",

```

```

        "F1 Micro": round(f1_score(y_test, y_pred, average='micro'), 4),
        "F1 Macro": round(f1_score(y_test, y_pred, average='macro'), 4),
        "Accuracy": round(accuracy_score(y_test, y_pred), 4),
        "PR-AUC": round(average_precision_score(y_test, y_scores, average='macro'), 4),
        "Tiempo (min)": str_tiempo,
        "Memoria (MB)": str_memoria
    })

del X_train_bert_scaled, X_test_bert_scaled, X_train_bert, X_test_bert
gc.collect()

```

```

--- MODELOS CON SCIBERT---
Modelo logístico OvR en Drive.
Modelo LinearSVC (OvR) en Drive.
Modelo SVC (RBF Aproximado) en Drive.
Modelo Random Forest en Drive.

--- MODELOS CON SCIBERT (Normalizado) ---
Modelo logístico OvR Norm en Drive.
Modelo MLP Norm en Drive.
48

```

✓ SciBERT con Fine-Tuning

```

print("--- MODELO SCIBERT CON FINE-TUNING ---")

num_cores = os.cpu_count() or 1

def multi_label_metrics(predictions, labels):
    # Eficiencia: Usamos scipy optimizado en C en lugar del puente Numpy->PyTorch->Numpy

    probs = scipy.special.expit(predictions)
    y_pred = np.zeros(probs.shape)
    y_pred[probs >= 0.5] = 1

    f1_macro = f1_score(labels, y_pred, average="macro")
    pr_auc = average_precision_score(labels, probs, average="macro")
    return {'f1_macro': f1_macro, 'pr_auc': pr_auc}

def compute_metrics(p: EvalPrediction):
    preds = p.predictions[0] if isinstance(p.predictions, tuple) else p.predictions
    return multi_label_metrics(predictions=preds, labels=p.label_ids)

# TOKENIZADOR
model_name = "allenai/scibert_scivocab_uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True, max_length=256)

test_dataset = Dataset.from_dict({
    'text': X_test_trans,
    'labels': y_test.astype(float).tolist()
})

tokenized_test = test_dataset.map(tokenize_function, batched=True, num_proc=num_cores)

ruta_scibert = os.path.join(save_dir_base, 'scibert_matematicas_finetuned')

args = TrainingArguments(
    output_dir=ruta_scibert + "_checkpoints",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16, # Aumentado para mejor rendimiento
    gradient_accumulation_steps=2, # Simula un batch de 32 si tu GPU tiene poca VRAM
    per_device_eval_batch_size=32,
    num_train_epochs=4,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="f1_macro",
    fp16=True,
    # OPTIMIZACIONES DE HARDWARE:
    dataloader_num_workers=4, # Evita que la GPU espere a la CPU para recibir batches
    torch_compile=False,
    save_total_limit=1, # Evita llenar el disco guardando todos los epochs intermedios
)

```



```

# ENTRENAMIENTO

if os.path.exists(ruta_scibert):
    print("Modelo SciBERT Fine-Tuned encontrado en Drive.")
    model = AutoModelForSequenceClassification.from_pretrained(ruta_scibert)
    tokenizer = AutoTokenizer.from_pretrained(ruta_scibert)
    trainer = Trainer(model=model,
                      args=args,
                      eval_dataset=tokenized_test,
                      processing_class=tokenizer,
                      compute_metrics=compute_metrics)

    t_mod = 77.7
    mem_mod = 108.04

else:
    print("Entrenando modelo SciBERT Fine-Tuned.")

    train_dataset = Dataset.from_dict({
        'text': X_train_trans,
        'labels': y_train.astype(float).tolist()
    })

    tokenized_train = train_dataset.map(tokenize_function, batched=True, num_proc=num_cores)

    # Carga del modelo base
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=29,
        problem_type="multi_label_classification"
    )

    trainer = Trainer(
        model=model,
        args=args,
        train_dataset=tokenized_train,
        eval_dataset=tokenized_test,
        processing_class=tokenizer,
        compute_metrics=compute_metrics
    )

    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    trainer.train()

    # Guardar modelo y tokenizador
    trainer.save_model(ruta_scibert)
    model.save_pretrained(ruta_scibert)
    tokenizer.save_pretrained(ruta_scibert)

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo SciBERT FT guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# PREDICCIÓN Y EVALUACIÓN
predicciones_brutas = trainer.predict(tokenized_test)
logits = predicciones_brutas.predictions

# Eficiencia: Evitamos mandar a Tensor y devolver a Numpy
probs = scipy.special.expit(logits)
y_pred_finetuned = (probs >= 0.5).astype(int)

resultados_globales.append({
    "Embedding": "SciBERT (Fine-Tuned)",
    "Modelo": "Red Neuronal",
    "F1 Micro": round(f1_score(y_test, y_pred_finetuned, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred_finetuned, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred_finetuned), 4),
    "PR-AUC": round(average_precision_score(y_test, probs, average='macro'), 4),
    "Tiempo (min)": t_mod,
    "Memoria (MB)": mem_mod
})

```

--- MODELO SCIBERT CON FINE-TUNING ---

config.json: 100%

385/385 [00:00<00:00, 44.7kB/s]

vocab.txt: 100%

228k/228k [00:00<00:00, 18.9MB/s]

Map (num_proc=12): 100%

93170/93170 [00:24<00:00, 4681.90 examples/s]

Modelo SciBERT Fine-Tuned encontrado en Drive.

Loading weights: 100%

201/201 [00:04<00:00, 95.19it/s, Materializing param=classifier.weight]

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Cargamos el modelo fine-tuned y su tokenizador
model_scibert_ft = AutoModelForSequenceClassification.from_pretrained(
    ruta_scibert,
    output_hidden_states=True # Para poder extraer los embeddings
).to(device)
tokenizer_scibert = AutoTokenizer.from_pretrained(ruta_scibert)
model_scibert_ft.eval()

# Extraemos los embeddings usando la función existente
X_test_scibert_ft = extract_finetuned_embeddings(X_test_trans, model_scibert_ft, tokenizer_scibert)

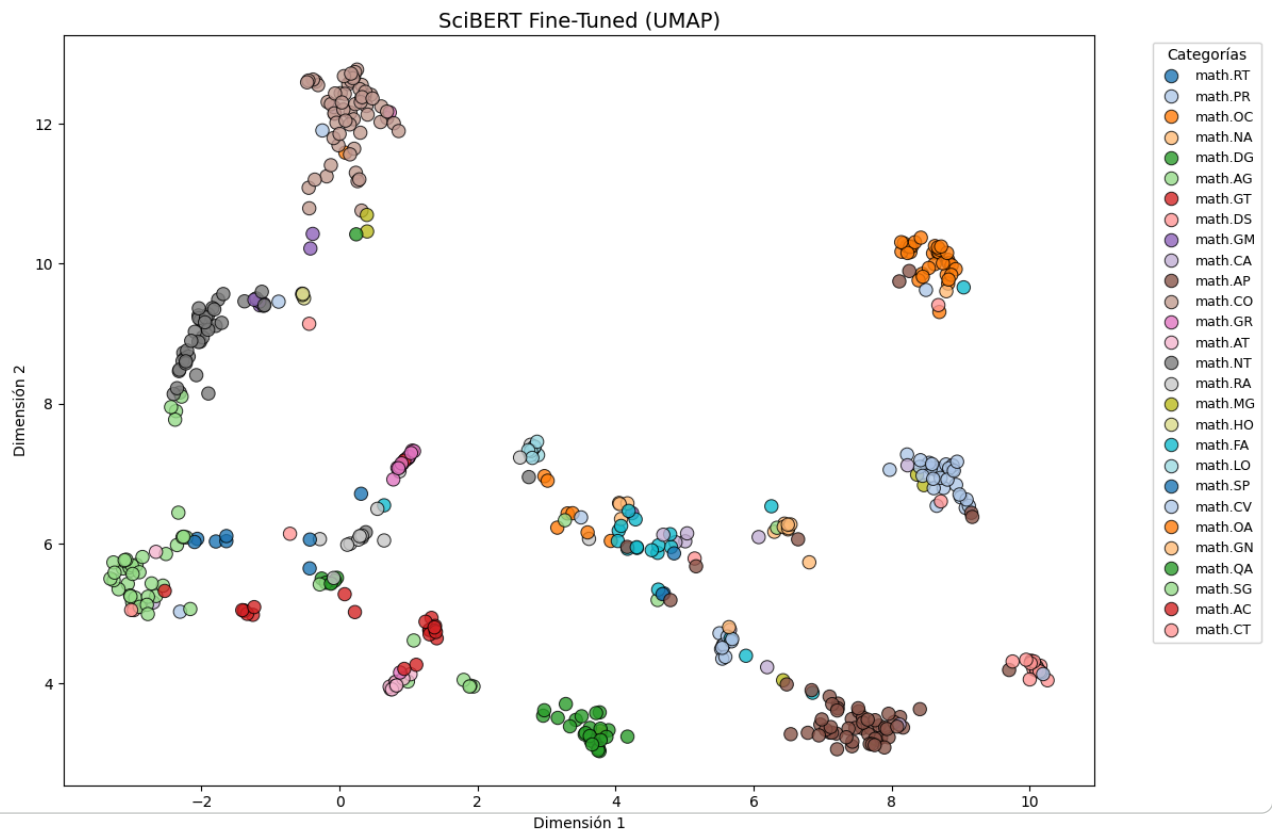
# Visualizamos con UMAP
visualizar_espacio_latente(
    X=X_test_scibert_ft,
    Y=y_test,
    clases=clases_matematicas,
    titulo="SciBERT Fine-Tuned",
    n_samples=500,
    metodo='umap'
)

# Limpiamos RAM/VRAM
del model_scibert_ft
torch.cuda.empty_cache()
```

Loading weights: 100%

201/201 [00:00<00:00, 928.43it/s, Materializing param=classifier.weight]

Extrayendo Embeddings: 100% [██████████] 2912/2912 [05:18<00:00, 9.13it/s]



MathBERT

```
print("--- MODELO MATHBERT CON FINE-TUNING ---")
```

```
num_cores = os.cpu_count() or 1
```

```
def multi_label_metrics(predictions, labels):
```

```

# Usamos scipy optimizado en C en lugar del puente Numpy->PyTorch->Numpy

probs = scipy.special.expit(predictions)
y_pred = np.zeros(probs.shape)
y_pred[probs >= 0.5] = 1

f1_macro = f1_score(labels, y_pred, average="macro")
pr_auc = average_precision_score(labels, probs, average="macro")
return {'f1_macro': f1_macro, 'pr_auc': pr_auc}

def compute_metrics(p: EvalPrediction):
    preds = p.predictions[0] if isinstance(p.predictions, tuple) else p.predictions
    return multi_label_metrics(predictions=preds, labels=p.label_ids)

# TOKENIZADOR
model_name = "tbs17/MathBERT"
tokenizer = AutoTokenizer.from_pretrained(model_name)

def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True, max_length=256)

test_dataset = Dataset.from_dict({
    # 'text': X_test_original,
    'text': X_test_trans,
    'labels': y_test.astype(float).tolist()
})

tokenized_test = test_dataset.map(tokenize_function, batched=True, num_proc=num_cores)

# ENTRENAMIENTO
ruta_mathbert = os.path.join(save_dir_base, 'mathbert_matematicas_finetuned')

args_mathbert = TrainingArguments(
    output_dir=ruta_mathbert + "_checkpoints",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16, # Aumentado para mejor rendimiento
    gradient_accumulation_steps=2, # Simula un batch de 32 si tu GPU tiene poca VRAM
    per_device_eval_batch_size=16,
    num_train_epochs=4,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="f1_macro",
    fp16=True,
    # OPTIMIZACIONES DE HARDWARE:
    dataloader_num_workers=4, # Evita que la GPU espere a la CPU para recibir batches
    torch_compile=False,
    save_total_limit=1,      # Evita llenar el disco guardando todos los epochs intermedios
)

if os.path.exists(ruta_mathbert):
    print("Modelo MathBERT Fine-Tuned encontrado en Drive.")
    model = AutoModelForSequenceClassification.from_pretrained(ruta_mathbert)
    tokenizer = AutoTokenizer.from_pretrained(ruta_mathbert)
    trainer = Trainer(model=model,
                      args=args_mathbert,
                      eval_dataset=tokenized_test,
                      processing_class=tokenizer,
                      compute_metrics=compute_metrics)

    t_mod = 81.04
    mem_mod = 4.78

else:
    print("Entrenando modelo MathBERT Fine-Tuned.")

    train_dataset = Dataset.from_dict({
        'text': X_train_trans,
        'labels': y_train.astype(float).tolist()
    })

    tokenized_train = train_dataset.map(tokenize_function, batched=True, num_proc=num_cores)

    # Carga del modelo base
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=29,
        problem_type="multi_label_classification"
    )

```

```

trainer = Trainer(
    model=model,
    args=args_mathbert,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    processing_class=tokenizer,
    compute_metrics=compute_metrics
)

gc.collect()
t0_mod = time()
mem0_mod = get_memory()

trainer.train()

# Guardar modelo y tokenizador
trainer.save_model(ruta_mathbert)
model.save_pretrained(ruta_mathbert)
tokenizer.save_pretrained(ruta_mathbert)

t_mod = round((time() - t0_mod) / 60, 2)
mem_mod = round(get_memory() - mem0_mod, 2)
print(f"Modelo SciBERT FT guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# PREDICCIÓN Y EVALUACIÓN
predicciones_brutas = trainer.predict(tokenized_test)
logits = predicciones_brutas.predictions

# Eficiencia: Evitamos mandar a Tensor y devolver a Numpy
probs = scipy.special.expit(logits)
y_pred_MATHBERT_finetuned = (probs >= 0.5).astype(int)

resultados_globales.append({
    "Embedding": "MathBERT (FineTuned)",
    "Modelo": "Red Neuronal",
    "F1 Micro": round(f1_score(y_test, y_pred_MATHBERT_finetuned, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred_MATHBERT_finetuned, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred_MATHBERT_finetuned), 4),
    "PR-AUC": round(average_precision_score(y_test, probs, average='macro'), 4),
    "Tiempo (min)": t_mod,
    "Memoria (MB)": mem_mod
})

```

```

--- MODELO MATHBERT CON FINE-TUNING ---
config.json: 100%          569/569 [00:00<00:00, 65.9kB/s]
tokenizer_config.json: 100%          28.0/28.0 [00:00<00:00, 3.44kB/s]
vocab.txt: 100%          232k/232k [00:00<00:00, 19.3MB/s]
tokenizer.json: 100%          466k/466k [00:00<00:00, 27.1MB/s]
Map (num_proc=12): 100%          93170/93170 [00:25<00:00, 3487.48 examples/s]
Modelo MathBERT Fine-Tuned encontrado en Drive.
Loading weights: 100%          201/201 [00:04<00:00, 41.67it/s, Materializing param=classifier.weight]

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_mathbert_ft = AutoModelForSequenceClassification.from_pretrained(ruta_mathbert,
                                                                    output_hidden_states=True # Para sacar los embeddings
                                                                    ).to(device)

tokenizer = AutoTokenizer.from_pretrained(ruta_mathbert)
model_mathbert_ft.eval()
X_test_mathbert_ft = extract_finetuned_embeddings(X_test_trans, model_mathbert_ft, tokenizer)

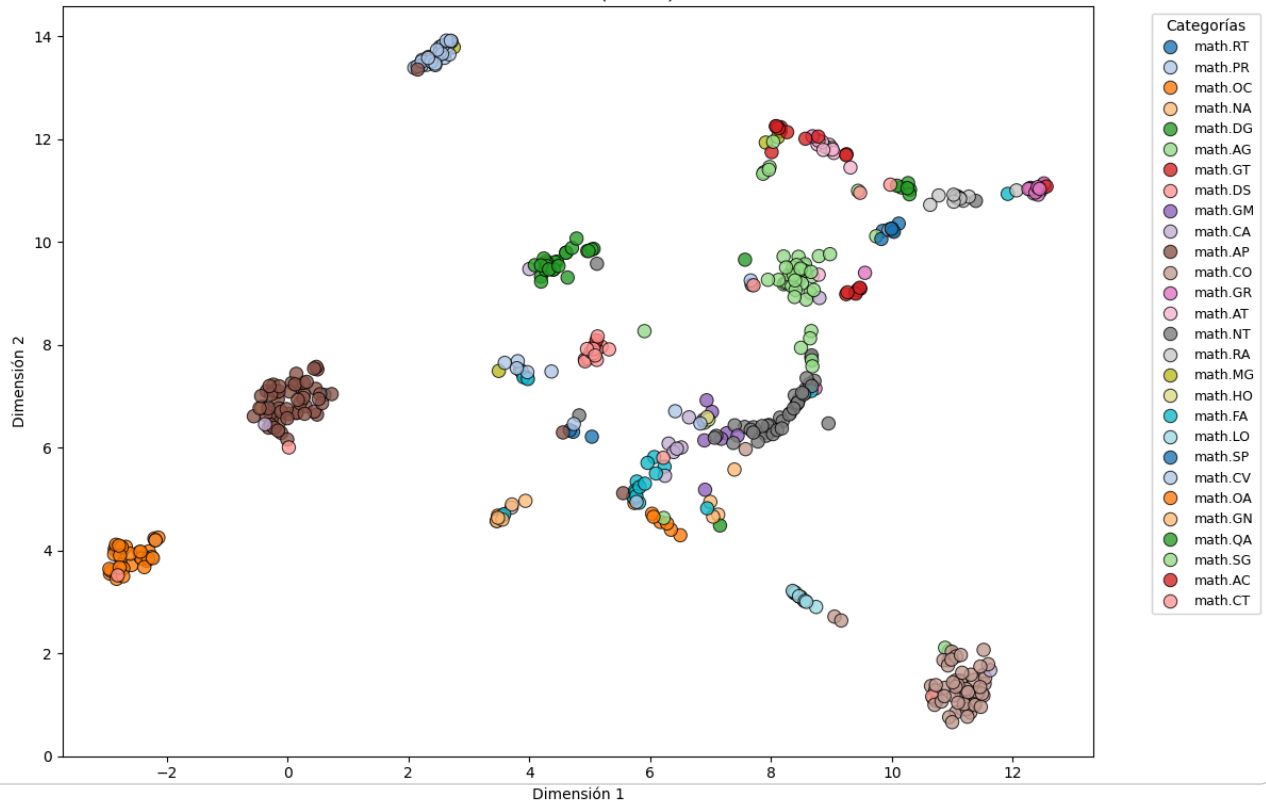
visualizar_espacio_latente(
    X=X_test_mathbert_ft,
    Y=y_test,
    clases=clases_matematicas,
    titulo="MATHBERT",
    n_samples=500,
    metodo='umap'
)

```

Loading weights: 100% 201/201 [00:00<00:00, 901.98it/s, Materializing param=classifier.weight]

Extrayendo Embeddings: 100% 2912/2912 [05:27<00:00, 8.89it/s]

MATHBERT (UMAP)



✓ T5 (encoder-decoder)

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

```
print("--- MODELO T5 CON FINE-TUNING ---")
```

```
def compute_metrics(p: EvalPrediction):
    # Asegurar que tomamos los logits si predictions es una tupla
    preds = p.predictions[0] if isinstance(p.predictions, tuple) else p.predictions
    probs = torch.sigmoid(torch.from_numpy(preds)).numpy()
    y_pred = (probs >= 0.5).astype(int)

    return {
        'f1_macro': f1_score(p.label_ids, y_pred, average="macro"),
        'accuracy': accuracy_score(p.label_ids, y_pred),
        'pr_auc': average_precision_score(p.label_ids, probs, average="macro")
    }
```

```
model_name = "t5-small"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
def tokenize_function(examples):
    # T5 suele funcionar bien con 256 o 512 tokens.
    return tokenizer(examples["text"], padding="max_length", truncation=True, max_length=256)
```

```
df_test = pd.DataFrame({'text': X_test_trans.tolist(), 'labels': y_test.astype(float).tolist()})
test_dataset = Dataset.from_pandas(df_test)
tokenized_test = test_dataset.map(tokenize_function, batched=True)
```

```
# ENTRENAMIENTO
```

```
ruta_t5 = os.path.join(save_dir_base, 't5_matematicas_finetuned')
```

```
args_t5 = TrainingArguments(
    output_dir=ruta_t5 + "_checkpoints",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=3e-4,
```

```
# --- ACELERADORES ---
```

```
per_device_train_batch_size=128,
per_device_eval_batch_size=128,
```

```

eval_accumulation_steps=10,      # Vaciado de memoria en evaluación
dataloading_num_workers=4,      # Usa 4 núcleos de CPU para cargar datos
# -----

num_train_epochs=4,
weight_decay=0.01,
load_best_model_at_end=True,
metric_for_best_model="f1_macro",
bf16=True,                      # Sigue usando bf16
tf32=True                      # Acelera aún más las operaciones matemáticas en GPUs Ampere (A100)
)

if os.path.exists(ruta_t5):
    print("Modelo T5 Fine-Tuned en Drive.")
    tokenizer = AutoTokenizer.from_pretrained(ruta_t5)
    model_t5 = AutoModelForSequenceClassification.from_pretrained(ruta_t5)

    trainer_t5 = Trainer(
        model=model_t5,
        args = args_t5,
        eval_dataset=tokenized_test,
        processing_class=tokenizer,
        compute_metrics=compute_metrics
    )

    t_mod = 99.39
    mem_mod = 1072.44

else:
    print("Entrenando modelo T5 Fine-Tuned.")

    # TOKENIZACIÓN
    df_train = pd.DataFrame({'text': X_train_trans.tolist(), 'labels': y_train.astype(float).tolist()})
    train_dataset = Dataset.from_pandas(df_train)
    tokenized_train = train_dataset.map(tokenize_function, batched=True)

    # HF adapta la arquitectura T5 para clasificación multietiqueta automáticamente
    model_t5 = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=29,
        problem_type="multi_label_classification"
    )

    trainer_t5 = Trainer(
        model=model_t5,
        args=args_t5,
        train_dataset=tokenized_train,
        eval_dataset=tokenized_test,
        processing_class=tokenizer,
        compute_metrics=compute_metrics
    )

    gc.collect()
    t0_mod = time()
    mem0_mod = get_memory()

    trainer_t5.train()
    trainer_t5.save_model(ruta_t5)
    model_t5.save_pretrained(ruta_t5)
    tokenizer.save_pretrained(ruta_t5)
    print("Modelo T5 guardado en Drive.")

    t_mod = round((time() - t0_mod) / 60, 2)
    mem_mod = round(get_memory() - mem0_mod, 2)
    print(f"Modelo T5 FT guardado en Drive. Tiempo: {t_mod} min | Memoria: {mem_mod} MB")

# PREDICCIÓN Y EVALUACIÓN FINAL
predicciones_brutas = trainer_t5.predict(tokenized_test)
preds_finales = predicciones_brutas.predictions[0] if isinstance(predicciones_brutas.predictions, tuple) else predicciones_brutas

# Convertir logits a probabilidades y luego a 1s y 0s
probs = torch.sigmoid(torch.from_numpy(preds_finales)).numpy()
y_pred_t5 = (probs >= 0.5).astype(int)

resultados_globales.append({
    "Embedding": "T5 (Fine-Tuned)",
    "Modelo": "Red Neuronal",
    "F1 Micro": round(f1_score(y_test, y_pred_t5, average='micro'), 4),
    "F1 Macro": round(f1_score(y_test, y_pred_t5, average='macro'), 4),
    "Accuracy": round(accuracy_score(y_test, y_pred_t5), 4),
})

```

```

"PR-AUC": round(average_precision_score(y_test, probs, average='macro'), 4),
"Tiempo (min)": t_mod,
"Memoria (MB)": mem_mod
})

```

```

--- MODELO T5 CON FINE-TUNING ---

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
 config.json: 100% 1.21k/1.21k [00:00<00:00, 128kB/s]

tokenizer_config.json: 100% 2.32k/2.32k [00:00<00:00, 293kB/s]

spiece.model: 100% 792k/792k [00:00<00:00, 1.30MB/s]

tokenizer.json: 100% 1.39M/1.39M [00:00<00:00, 5.56MB/s]

Map: 100% 93170/93170 [00:16<00:00, 5661.98 examples/s]

Entrenando modelo T5 Fine-Tuned.

Map: 100% 375713/375713 [01:05<00:00, 6591.61 examples/s]

model.safetensors: 100% 242M/242M [00:01<00:00, 168MB/s]

Loading weights: 100% 131/131 [00:00<00:00, 835.10it/s, Materializing param=transformer.shared.weight]

T5ForSequenceClassification LOAD REPORT from: t5-small

Key	Status
classification_head.dense.weight	MISSING
classification_head.out_proj.weight	MISSING
classification_head.out_proj.bias	MISSING
classification_head.dense.bias	MISSING

Notes:

- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
 [11744/11744 1:39:19, Epoch 4/4]

Epoch	Training Loss	Validation Loss	F1 Macro	Accuracy	Pr Auc
1	0.077801	0.074955	0.597192	0.508898	0.663134
2	0.068890	0.069185	0.635309	0.534915	0.703046
3	0.063839	0.066865	0.650024	0.546056	0.717940
4	0.061074	0.066040	0.656199	0.550166	0.723042

Writing model shards: 100% 1/1 [00:00<00:00, 1.64it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 1.57it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 1.61it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 1.65it/s]

There were missing keys in the checkpoint model loaded: ['transformer.encoder.embed_tokens.weight', 'transformer.decoder.emt

Writing model shards: 100% 1/1 [00:00<00:00, 1.63it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 1.55it/s]

Modelo T5 guardado en Drive.

Modelo SciBERT FT guardado en Drive. Tiempo: 99.39 min | Memoria: 1072.44 MB

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

model_t5_ft = AutoModelForSequenceClassification.from_pretrained(ruta_t5).to(device)
tokenizer_t5 = AutoTokenizer.from_pretrained(ruta_t5)
model_t5_ft.eval()

```

```

X_test_t5_ft = extract_t5_embeddings(X_test_trans, model_t5_ft, tokenizer_t5)

```

```

visualizar_espacio_latente(
    X=X_test_t5_ft,
    Y=y_test,
    clases=clases_matematicas,
    titulo="T5 Fine-Tuned",
    n_samples=500,
    metodo='umap'
)

```

```

# Limpiamos RAM/VRAM
del model_t5_ft
torch.cuda.empty_cache()

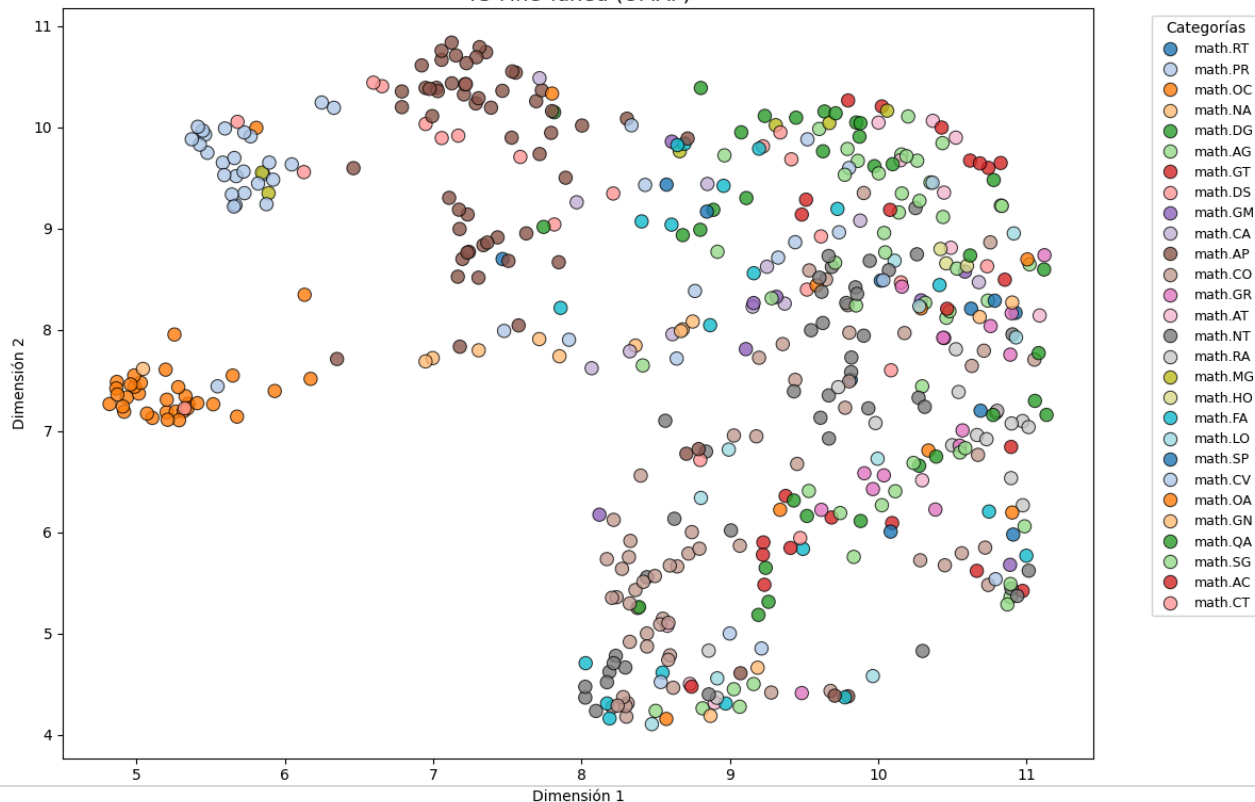
```

Loading weights: 100%

135/135 [00:00<00:00, 689.14it/s, Materializing param=transformer.shared.weight]

Extrayendo T5: 100% [██████████] 2912/2912 [01:07<00:00, 43.06it/s]

T5 Fine-Tuned (UMAP)



COMPARATIVA

```
print("\n--- RESULTADOS FINALES DE LOS MODELOS ---")

# Convertir la lista de diccionarios a un DataFrame
df_resultados = pd.DataFrame(resultados_globales)

# Ordenar los resultados por el mejor F1 Macro
df_resultados = df_resultados.sort_values(by="F1 Macro", ascending=False).reset_index(drop=True)

# Mostrar la tabla
display(df_resultados)
```


--- RESULTADOS FINALES DE LOS MODELOS ---

	Embedding	Modelo	F1 Micro	F1 Macro	Accuracy	PR-AUC	Tiempo (min)	Memoria (MB)
0	MathBERT (FineTuned)	Red Neuronal	0.7611	0.6980	0.5795	0.7608	81.04	4.78
1	SciBERT (Fine-Tuned)	Red Neuronal	0.7592	0.6979	0.5782	0.7639	77.7	108.04
2	T5 (Fine-Tuned)	Red Neuronal	0.7330	0.6562	0.5502	0.7230	99.39	1072.44
3	TF-IDF	MLP	0.7306	0.6556	0.5377	0.7223	1.35 + 37.53 = 38.88	28119.14 + -738.69 = 27380.45
4	Word2Vec (Norm)	MLP	0.7134	0.6305	0.5285	0.7093	3.78 + 12.89 = 16.67	3819.23 + 1.15 = 3820.38

COMENTARIOS

- 5 Word2Vec (Norm) RegLog (OvR) 0.6687 0.5680 0.4721 0.6538 3.78 + 1.51 = 5.29 3819.23 + 11.28 = 3830.51
- Los transformers con finetuning son los ganadores. En particular, MathBERT, probablemente gracias a que fue entrenado en inglés, con fórmulas matemáticas y a saber relaciones... 0.6844... 0.5569 0.4541 0.6306 3.78 + 2.2 = 5.98 3819.23 + 26.62 = 3845.85
 - También destaca la diferencia entre sciBERT con finetuning y sin él. Extraer solo los embeddings no es comparable a reajustar todos sus pesos internos para esta tarea específica. 0.6877 0.5273 0.2303 0.6647 1.35 + 13.29 = 14.64 28119.14 + 1.75 = 28120.89
 - Detrás de los transformers está TF-IDF que supera a técnicas más modernas como Word2Vec o GloVe. Probablemente porque TF-IDF es capaz de detectar vocabulario (más específico de cada categoría) como "grafo" por ejemplo, mientras que GloVe o Word2Vec al tener que promediar los embeddings para obtener una representación del abstract, esta información se diluye. 1.35 + 25.68 = 27.03 28119.14 + 1.07 = 28120.21
 - 6 SciBERT (Norm) MLP 0.5607 0.4327 0.3921 0.5324 22.37 + 69.62 = 91.99 1739.41 + 12.96 = 1752.37
 - Los peores modelos son con LinearSVC y RegLog, probablemente por lo mismo, al promediar los vectores en Word2Vec y GloVe, los conceptos se pierden. No se puede separar las categorías trazando hiperplanos.
 - 7 GloVe (Norm) MLP 0.5316 0.4074 0.3637 0.5032 0.78 + 9.93 = 10.71 1840.8 + -4.19 = 1836.61
 - Word2Vec mejora gracias a usar MLP, el modelo fue capaz de encontrar patrones complejos (no lineales).
 - 8 SciBERT RegLog (OvR) 0.5185 0.4036 0.3370 0.4909 22.37 + 20 = 42.37 1739.41 + -46.11 = 1693.3

Métrica usada

- 12 Word2Vec LinearSVC (OvR) 0.4839 0.4380 0.1197 0.6058 3.78 + 21.65 = 25.43 3819.23 + -48.27 = 3770.96
- He elegido F1-macro sobre accuracy (que tiene peores valores), ya que obtener un accuracy alto es más complicado en un problema multietiqueta. Acertar exactamente las mismas categorías que la base de datos no es fácil. Por eso, F1-macro es más realista, permitiendo valorar aciertos parciales, es decir, acertar en alguna categoría de todas las que hay de un abstract.
 - 13 Word2Vec SVC (RBF Aproximado) 0.4772 0.4403 0.1408 0.6003 3.78 + 49.05 = 52.83 3819.23 + 355.66 = 4174.91
 - 14 SciBERT (Norm) RegLog (OvR) 0.4409 0.2855 0.2777 0.4135 22.37 + 3.75 = 26.12 1739.41 + 47.13 = 1786.54

✓ EJEMPLOS

```
np.random.seed(42)

print("--- EJEMPLOS DEL: TF-IDF + MLP ---")
X_test_ejemplo = scipy.sparse.load_npz(ruta_test_tfidf)

# El modelo
y_pred_best = mlp_tfidf.predict(X_test_ejemplo)

# Todas las categorías
nombres_categorias = mlb.classes_.tolist()

# Listas
indices_aciertos_perfectos = []
indices_errores = []

for i in range(len(y_test)):
    "
```